

RECURSIVE FUNCTIONS AND MODELS OF COMPUTATION

1. INTRODUCTION

The goal of these notes is to formalize what constitutes an effective procedure. For this we introduce several models for computation. We consider models of two kinds: abstract and mechanical. Abstract models consist of partial recursive functions and functions defined by existential rudimentary formulas of the first-order language of arithmetic. Among the mechanical models we have Loop-while programs, register machines and Turing-Post machines. We prove that they are all equivalent in expressive power. Moreover, all notions introduced in these notes are formulated from the beginning in a way that naturally relativizes to oracles.

2. PARTIAL FUNCTIONS AND RELATIONS ON POWERS OF NATURAL NUMBERS

This section introduces terminology used throughout the notes.

The set of all natural numbers is $\mathbb{N}$. Note that $\mathbb{N}$ is well ordered by the standard relation $\leq$. By an initial interval of $\mathbb{N}$ we mean an initial segment with respect to $\leq$.

If $k \in \mathbb{N}$, then the set of k -tuples of numbers is the Cartesian product $\mathbb{N}^k$. We denote k -tuples of numbers by vector notation $\vec{n}$ and refer to the number k as the length of $\vec{n}$. In the case $k = 0$ the set $\mathbb{N}^0$ contains a unique element: the empty tuple.

Definition 2.1. Let $k \in \mathbb{N}$. Let f be a function with domain a subset of $\mathbb{N}^k$ and taking values in $\mathbb{N}$. Then f is a k -ary partial function.

Definition 2.2. A partial function is a k -ary partial function for some $k \in \mathbb{N}$.

Definition 2.3. Let f be a k -ary partial function. If $\vec{n} \in \mathbb{N}^k$ is in the domain of f , then we say that f is defined at $\vec{n}$.

Remark 2.4. Let f be a k -ary partial function and let $\vec{n} \in \mathbb{N}^k$. We write $f(\vec{n}) \downarrow$ to denote that f is defined at $\vec{n}$. If f is not defined at $\vec{n}$, then we use $f(\vec{n}) \uparrow$.

Definition 2.5. Let $k \in \mathbb{N}$ and let f be a k -ary partial function. Then

$$\Gamma_f = \{ (\vec{n}, f(\vec{n})) \in \mathbb{N}^{k+1} \mid \text{for } \vec{n} \in \mathbb{N}^k \text{ such that } f(\vec{n}) \downarrow \}$$

is the graph of f .

Definition 2.6. Let f be a k -ary partial function such that $f(\vec{n}) \downarrow$ for every $\vec{n} \in \mathbb{N}^k$. Then f is called a k -ary total function.

Definition 2.7. Let g be a k -ary partial function and let $f_1, \dots, f_k$ be l -ary partial functions. Consider all $\vec{n} \in \mathbb{N}^l$ such that $f_1(\vec{n}) \downarrow, \dots, f_k(\vec{n}) \downarrow$ and $g(f_1(\vec{n}), \dots, f_k(\vec{n})) \downarrow$. Then the map

$$\vec{n} \mapsto g(f_1(\vec{n}), \dots, f_k(\vec{n}))$$

gives rise to a well defined l -ary partial function. We refer to this partial function as the *composition of g with $f_1, \dots, f_k$* .

Definition 2.8. Let g be a $(k+2)$ -ary partial function and let f be a k -ary partial function for some $k \in \mathbb{N}$. Fix $\vec{n} \in \mathbb{N}^k$. We define $h(\vec{n}, m)$ for numbers m in some initial interval of $\mathbb{N}$. We set

$$h(\vec{n}, 0) = f(\vec{n})$$

if $f(\vec{n}) \downarrow$. Now suppose that $h(\vec{n}, m) \downarrow$ for some number m and that $g(\vec{n}, m, h(\vec{n}, m)) \downarrow$. Then we set

$$h(\vec{n}, m+1) = g(\vec{n}, m, h(\vec{n}, m))$$

Then h is a $(k+1)$ -ary partial function obtained by *primitive recursion* from g and f .

Definition 2.9. Let g be a $(k+1)$ -ary partial function for some $k \in \mathbb{N}$. Fix $\vec{n} \in \mathbb{N}^k$. Suppose that there exists number l such that

$$g(\vec{n}, 0) \downarrow, g(\vec{n}, 1) \downarrow, \dots, g(\vec{n}, l-1) \downarrow$$

are defined, nonzero numbers and $g(\vec{n}, l) = 0$. Then we define $\mu_m[g(\vec{n}, m) = 0] = l$. If such number l does not exist, then $\mu_m[g(\vec{n}, m) = 0]$ is undefined. This gives rise to a k -ary partial function $\mu_m[g(\vec{n}, m) = 0]$ called the *unbounded minimization* of g .

We also need some terminology concerning relations.

Definition 2.10. Let $k \in \mathbb{N}$ and let $R \subseteq \mathbb{N}^k$. Then R is a *k-ary relation*.

Definition 2.11. Let $k \in \mathbb{N}$ and let R be a k -ary relation. The k -ary function

$$\mathbb{1}_R(\vec{n}) = \begin{cases} 1 & \text{if } \vec{n} \in R \\ 0 & \text{otherwise} \end{cases}$$

is called the *characteristic function* of R .

Definition 2.12. Let R be a k -ary relation. Let c_R be partial function such that $c_R(\vec{n}) = 1$ for all $\vec{n} \in R$ and $c_R(\vec{n}) \uparrow$ otherwise. Then c_R is the *semicharacteristic function* of R .

Next we introduce some notions concerning class of functions.

Definition 2.13. Let $\mathcal{F}$ be a class of partial functions.

- (1) Suppose that for every positive $k \in \mathbb{N}$, every k -ary partial function $g \in \mathcal{F}$ and all partial functions $f_1, \dots, f_k \in \mathcal{F}$ of the same arity the composition of g with $f_1, \dots, f_k$ is in $\mathcal{F}$. Then $\mathcal{F}$ is *closed under composition*.
- (2) Suppose that for every positive $k \in \mathbb{N}$, every $(k+2)$ -ary partial function $g \in \mathcal{F}$ and every k -ary partial function $f \in \mathcal{F}$ the function obtained by primitive recursion from g and f is in $\mathcal{F}$. Then $\mathcal{F}$ is *closed under primitive recursion*.
- (3) Suppose that for every positive $k \in \mathbb{N}$ and every $(k+1)$ -ary partial function $g \in \mathcal{F}$ the function $\mu_m[g(\vec{n}, m) = 0]$ is in $\mathcal{F}$. Then $\mathcal{F}$ is *closed under unbounded minimization*.

Finally, we use the following notation. For every $k \in \mathbb{N}$ we denote by Z_k the function given by formula $\vec{n} \mapsto 0$ for each $\vec{n} \in \mathbb{N}^k$. We denote by s the successor function i.e. $s(n) = n+1$ for $n \in \mathbb{N}$. For every positive $k \in \mathbb{N}$ and each $i \in \mathbb{N}$ such that $1 \leq i \leq k$ we denote by I_k^i the projection $\vec{n} \mapsto n_i$ where $\vec{n} = (n_1, \dots, n_k) \in \mathbb{N}^k$.

3. PRIMITIVE RECURSIVE FUNCTIONS

Definition 3.1. The class of *primitive recursive functions* is the smallest class of partial functions that contains:

- (1) the successor s ,
- (2) Z_k for every $k \in \mathbb{N}$,
- (3) I_k^i for every positive $k \in \mathbb{N}$ and $1 \leq i \leq k$,

and is closed under composition and primitive recursion.

Proposition 3.2. *All primitive recursive functions are total.*

Proof. Note that the class of total functions is closed under composition and primitive recursion. Since the class of primitive recursive functions is defined by these operations, it follows that all primitive recursive functions are total. $\square$

The remainder of this section is devoted to proving that most well-known total functions are primitive recursive.

Proposition 3.3. *Then all k -ary total constant functions are primitive recursive for every $k \in \mathbb{N}$.*

Proof. Let j be a number. By composing the successor function with itself j -times we obtain the function $n \mapsto n + j$, which is primitive recursive. Next by composing this function with the zero k -ary function, we derive the k -ary total constant function with j as the only value. Hence this function is also primitive recursive. This completes the proof. $\square$

Proposition 3.4. *The addition function $(n, m) \mapsto n + m$ is primitive recursive.*

Proof. The identity function is primitive recursive. Similarly the function $(n, m, l) \mapsto l + 1$ is primitive recursive. Next the function $(n, m) \mapsto n + m$ is obtained by primitive recursion from the two functions described above. $\square$

Proposition 3.5. *The multiplication function $(n, m) \mapsto n \cdot m$ is primitive recursive.*

Proof. Note that the function $(n, m, l) \mapsto n + l$ is the composition of addition function with projections on first and last factor. Hence it is primitive recursive. Next observe that the function $(n, m) \mapsto n \cdot m$ is obtained by primitive recursion from $(n, m, l) \mapsto n + l$ and the zero unary function. $\square$

Proposition 3.6. *The exponential function $(n, m) \mapsto n^m$ is primitive recursive.*

Proof. Note that the function $(n, m, l) \mapsto n \cdot l$ is the composition of multiplication with projections on first and last factors. Hence it is primitive recursive by Proposition 3.5. Now the function $(n, m) \mapsto n^m$ is obtained by primitive recursion from $(n, m, l) \mapsto n \cdot l$ and constant unary function with value 1. Since both these functions are primitive recursive, the proof is completed. $\square$

Definition 3.7. The function

$$n \mapsto \begin{cases} n - 1 & \text{if } n > 0 \\ 0 & \text{otherwise} \end{cases}$$

is the *predecessor* function.

Proposition 3.8. *The predecessor function is primitive recursive.*

Proof. The predecessor function is obtained by primitive recursion from the 2-ary projection on the first factor and the zero nullary function. Hence it is primitive recursive. $\square$

Definition 3.9. The 2-ary function

$$n \dot{-} m = \begin{cases} n - m & \text{if } n > m \\ 0 & \text{otherwise} \end{cases}$$

is the *monus* function.

Proposition 3.10. *The monus function is primitive recursive.*

Proof. Note that the function $(n, m, l) \mapsto l \dot{-} 1$ is the composition of the predecessor with the projection on the last factor. By Proposition 3.8 it is primitive recursive. The monus function is obtained from $(n, m, l) \mapsto l \dot{-} 1$ and the identity by primitive recursion. Thus it is primitive recursive. $\square$

Proposition 3.11. *Let $k \in \mathbb{N}$ and let f be a $(k+1)$ -ary primitive recursive function. Then functions*

$$(\vec{n}, m) \mapsto \sum_{l < m} f(\vec{n}, l), (\vec{n}, m) \mapsto \prod_{l < m} f(\vec{n}, l)$$

are primitive recursive.

Proof. We prove the result for sum. The proof for product is analogous.

First observe that the function $(\vec{n}, m, l) \mapsto f(\vec{n}, m)$ is the composition of f and $I_1^{k+2}, \dots, I_{k+1}^{k+2}$. Hence it is primitive recursive. Next note that the function $(\vec{n}, m, l) \mapsto l + f(\vec{n}, m)$ is primitive recursive as the composition of addition with the function described above and I_{k+2}^{k+2} . Finally the function

$$(\vec{n}, m) \mapsto \sum_{l < m} f(\vec{n}, l)$$

is obtained by primitive recursion from the function described above and the k -ary zero function. $\square$

Corollary 3.12. *Let $k \in \mathbb{N}$ and let f be a $(k+1)$ -ary primitive recursive function. Then functions given by formulas*

$$(\vec{n}, m) \mapsto \sum_{l \leq m} f(\vec{n}, l), (\vec{n}, m) \mapsto \prod_{l \leq m} f(\vec{n}, l)$$

are primitive recursive.

Proof. Left for the reader as an exercise. $\square$

Definition 3.13. The function

$$\text{sign}(n) = \begin{cases} 1 & \text{if } n > 0 \\ 0 & \text{otherwise} \end{cases}$$

is the *sign* function.

Proposition 3.14. *The sign function is primitive recursive.*

Proof. The sign function is obtained by primitive recursion from the constant 2-ary function with value 1 and the nullary zero function. Since both these functions are primitive recursive, we obtain the statement. $\square$

4. PRIMITIVE RECURSIVE RELATIONS

Definition 4.1. Let R be a k -ary relation for some $k \in \mathbb{N}$. Suppose that the characteristic function of R is primitive recursive. Then R is *primitive recursive*.

Proposition 4.2. *Let $k \in \mathbb{N}$ and let R, S be k -ary primitive recursive relations. Then the following assertions hold.*

- (1) $R \cup S$ is primitive recursive.
- (2) $R \cap S$ is primitive recursive.
- (3) $\mathbb{N}^k \setminus R$ is primitive recursive.

Proof. For the proof of **(1)** note that the function obtained by applying sign to the sum $\mathbb{1}_R + \mathbb{1}_S$ is primitive recursive. Since this function coincides with $\mathbb{1}_{R \cup S}$, we derive that **(1)** holds.

Since $\mathbb{1}_{R \cap S} = \mathbb{1}_R \cdot \mathbb{1}_S$, we deduce that **(2)** holds.

Let $\mathbb{1}_{\mathbb{N}^k}$ be a constant k -ary function with value 1. By composing monus function with $\mathbb{1}_{\mathbb{N}^k}, \mathbb{1}_R$ we deduce that

$$\mathbb{1}_{\mathbb{N}^k \setminus R} = \mathbb{1}_{\mathbb{N}^k} \dot{-} \mathbb{1}_R$$

is primitive recursive. This proves **(3)**. $\square$

Proposition 4.3. *Let $k \in \mathbb{N}$ and let R be a $(k+1)$ -ary primitive recursive relation. Then*

$$\{(\vec{n}, m) \in \mathbb{N}^{k+1} \mid (\vec{n}, l) \in R \text{ for all } l < m\}, \{(\vec{n}, m) \in \mathbb{N}^{k+1} \mid (\vec{n}, l) \in R \text{ for all } l \leq m\},$$

$$\{(\vec{n}, m) \in \mathbb{N}^{k+1} \mid (\vec{n}, l) \in R \text{ for some } l < m\}, \{(\vec{n}, m) \in \mathbb{N}^{k+1} \mid (\vec{n}, l) \in R \text{ for some } l \leq m\}$$

are primitive recursive.

Proof. This follows immediately from Proposition 3.11 and Corollary 3.12 combined with the fact that sign is primitive recursive. $\square$

Proposition 4.4. *The binary relations $<, \leq, =$ are primitive recursive.*

Proof. The relation $<$ is primitive recursive because its characteristic function can be expressed by combination of monus and sign which are primitive recursive functions. By Proposition 4.2 the relation $\geq$ is primitive recursive as well. Since

$$\{(n, m) \in \mathbb{N}^2 \mid n \leq m\}$$

is obtained by swapping variables in

$$\{(m, n) \in \mathbb{N}^2 \mid n \geq m\}$$

it is also primitive recursive. Finally using Proposition 4.2 and the fact that intersection of relation above is the diagonal of $\mathbb{N}^2$ we derive the statement. $\square$

Now we prove some useful results regarding primitive recursive relations and functions.

Proposition 4.5. *Let $k, m \in \mathbb{N}$. Let $R_1, \dots, R_m$ be k -ary primitive recursive relations and let $f_1, \dots, f_m$ be k -ary primitive recursive functions. Suppose that*

$$\mathbb{N}^k = R_1 \cup \dots \cup R_m$$

is a decomposition onto pairwise disjoint sets. Then the k -ary function defined by

$$\vec{n} \mapsto \begin{cases} f_1(\vec{n}) & \text{if } \vec{n} \in R_1 \\ \dots & \\ f_m(\vec{n}) & \text{if } \vec{n} \in R_m \end{cases}$$

is primitive recursive.

Proof. Indeed, we have

$$f(\vec{n}) = f_1(\vec{n}) \cdot \mathbb{1}_{R_1}(\vec{n}) + \dots + f_m(\vec{n}) \cdot \mathbb{1}_{R_m}(\vec{n})$$

for every $\vec{n} \in \mathbb{N}^k$. $\square$

Definition 4.6. Let $k \in \mathbb{N}$ and let R be a $(k+1)$ -ary relation. We define

$$\mu_{l < m}[(\vec{n}, l) \in R] = \begin{cases} \min\{l \mid l < m \text{ and } (\vec{n}, l) \in R\} & \text{if } (\vec{n}, l) \in R \text{ for some } l < m \\ m & \text{otherwise} \end{cases}$$

Then $\mu_{l < m}[(\vec{n}, l) \in R]$ is said to be obtained by *bounded minimization* from R .

Finally we prove important result that bounded minimization of primitive recursive relation is primitive recursive function.

Proposition 4.7. Let $k \in \mathbb{N}$ and let R be a $(k+1)$ -ary primitive recursive relation. Then the function

$$(\vec{n}, m) \mapsto \mu_{l < m}[(\vec{n}, l) \in R]$$

is primitive recursive.

Proof. Define a relation

$$N = \{(\vec{n}, m) \in \mathbb{N}^k \mid (\vec{n}, l) \notin R \text{ for all } l < m\}$$

Then N is the $(k+1)$ -ary primitive recursive relation by Proposition 4.3. Next $S = N \cap R$ is a primitive recursive $(k+1)$ -ary relation such that $(\vec{n}, m) \in S$ if and only if m is the smallest number such that $(\vec{n}, m) \in R$. Now observe that

$$\mu_{l < m}[(\vec{n}, l) \in R] = m \cdot \mathbb{1}_N(\vec{n}, m) + \sum_{l < m} l \cdot \mathbb{1}_S(\vec{n}, l)$$

for every $\vec{n} \in \mathbb{N}^k$ and $m \in \mathbb{N}$. This function is primitive recursive by Proposition 3.11, the fact that sum is primitive recursive and our observations that N, S are primitive recursive $(k+1)$ -ary relations. $\square$

5. NUMBER THEORY AND PRIMITIVE RECURSION

In this section we focus on some elementary number-theoretic properties and their relation to primitive recursion.

Proposition 5.1. The divisibility relation $\mid$ is primitive recursive.

Proof. We have $n \mid m$ if and only if $l \cdot n = m$ for some $l \leq m$. Hence divisibility is primitive recursive by Proposition 4.3. $\square$

Proposition 5.2. The set $\mathbb{P}$ of prime numbers is primitive recursive.

Proof. We have

$$\mathbb{P} = \{p \in \mathbb{N} \mid p > 1 \text{ and } p \text{ is not divisible by } n \text{ for all } 1 < n < p\}$$

Hence $\mathbb{P}$ is primitive recursive by results of previous section. $\square$

Theorem 5.3. Let p_n denote the n -th prime number. Then the function $n \mapsto p_n$ is primitive recursive.

For the proof we need some elementary result.

Lemma 5.3.1. The factorial function $n \mapsto n!$ is primitive recursive.

Proof of the lemma. Left for the reader as an exercise. $\square$

Proof of the theorem. Consider the total 2-ary function

$$f(n, m) = \mu_{l < m} [(n < l) \text{ and } l \in \mathbb{P}]$$

According to Proposition 5.2 and Proposition 4.7 we infer that f is primitive recursive. By Lemma 5.3.1 we derive that

$$g(n) = f(n, n! + 2)$$

is primitive recursive. Note that $g(n)$ is the smallest prime number greater than n .

Now we have $p_0 = 2$ and

$$p_{n+1} = g(p_n)$$

for every n . Hence $n \mapsto p_n$ is obtained by primitive recursion from $g \cdot I_2^2$ and constant function equal to 2. Therefore, it is primitive recursive. $\square$

Proposition 5.4. For numbers n, m we define

$$e_m(n) = \begin{cases} \max \{l \mid p_m^l \text{ divides } n\} & \text{if } n > 0 \\ 0 & \text{if } n = 0 \end{cases}$$

Then the function $(n, m) \mapsto e_m(n)$ is primitive recursive.

Proof. Suppose that $n > 0$. Then

$$e_m(n) = \mu_{l < n} [n \text{ is not divisible by } p_m^l] \div 1$$

for all $m \in \mathbb{N}$. Clearly $e_m(0) = 0$ for all $m \in \mathbb{N}$. Hence results of the previous section and the fact that the monus is primitive recursive imply that $(n, m) \mapsto e_m(n)$ is primitive recursive. $\square$

6. PRIMITIVE RECURSIVE ENCODING OF SEQUENCES

In this section we explore how finite sequences of natural numbers can be represented and manipulated within the framework of primitive recursive functions. We will define encoding schemes for sequences and demonstrate that common operations on sequences are primitive recursive.

Definition 6.1. We define

$$[\vec{n}] = \begin{cases} p_0^{1+n_0} \cdot \dots \cdot p_k^{1+n_k} & \text{if } \vec{n} = (n_0, \dots, n_k) \in \mathbb{N}^{k+1} \text{ for some } k \\ 1 & \text{if } \vec{n} \text{ is empty tuple} \end{cases}$$

Then $[\vec{n}]$ is the *sequence number* of $\vec{n}$.

Proposition 6.2. Let Seq denote the set of sequence numbers of all tuples. Then Seq is a primitive recursive set.

Proof. We have

$$\text{Seq} = \left\{ n \in \mathbb{N} \mid n > 0 \text{ and for all } l < n \text{ either } e_l(n) = 0 \text{ or } e_l(n) \geq 1 \right\}$$

Hence Seq is primitive recursive. $\square$

Definition 6.3. We define

$$\text{lh}(n) = \begin{cases} k & \text{if } n = [\vec{n}] \text{ for some } \vec{n} \in \mathbb{N}^k \\ 0 & \text{otherwise} \end{cases}$$

Then $\text{lh}(n)$ is the *length* of n .

Proposition 6.4. The function $n \mapsto \text{lh}(n)$ is primitive recursive.

Proof. Consider set

$$\mathbb{P}_n = \{l \in \mathbb{N} \mid p_l \mid n\}$$

of indexes of prime divisors of n . Then function

$$n \mapsto \sum_{l < n} \mathbb{1}_{\mathbb{P}_n}(l)$$

is primitive recursive. Note that

$$\text{lh}(n) = \begin{cases} \sum_{l < n} \mathbb{1}_{\mathbb{P}_n}(l) & \text{if } n \in \text{Seq} \\ 0 & \text{otherwise} \end{cases}$$

Hence lh is primitive recursive and the proposition is proved. $\square$

Proposition 6.5. For numbers $n, l \in \mathbb{N}$ define

$$n[l] = \begin{cases} n_l & \text{if } n = [\vec{n}] \text{ for some } k \in \mathbb{N}, \vec{n} \in \mathbb{N}^{k+1} \text{ and } 0 \leq l \leq k \\ 0 & \text{otherwise} \end{cases}$$

Then the function $(n, l) \mapsto n[l]$ is primitive recursive.

Proof. We have

$$n[l] = \begin{cases} e_l(n) \div 1 & \text{if } n \in \text{Seq and } 0 \leq l < \text{lh}(n) \\ 0 & \text{otherwise} \end{cases}$$

and hence the proposition follows. $\square$

Proposition 6.6. Let $k \in \mathbb{N}$. Then the k -ary function

$$\mathbb{N}^k \ni \vec{n} \mapsto [\vec{n}] \in \mathbb{N}$$

is primitive recursive.

Proof. Left for the reader. $\square$

Suppose that $n, m \in \text{Seq}$. Then $n \star m$ is a sequence number such that

$$\text{lh}(n \star m) = \text{lh}(n) + \text{lh}(m)$$

and

$$(n \star m)[l] = \begin{cases} n[l] & \text{if } 0 \leq l < \text{lh}(n) \\ m[l - \text{lh}(n)] & \text{if } \text{lh}(n) \leq l < \text{lh}(n) + \text{lh}(m) \end{cases}$$

Moreover, if one of $n, m \in \mathbb{N}$ is not a sequence number, then we set $n \star m = 0$.

Proposition 6.7. The function $(n, m) \mapsto n \star m$ is primitive recursive.

Proof. Suppose that $n, m \in \text{Seq}$. Then

$$n \star m = n \cdot \prod_{l < \text{lh}(m)} p_{\text{lh}(n)+l}^{1+m[l]}$$

and hence $(n, m) \mapsto n \star m$ is primitive recursive. $\square$

7. RECURSIVE FUNCTIONS RELATIVE TO SUBSET

First we introduce the relative version of primitive recursive functions.

Definition 7.1. Let A be a subset of $\mathbb{N}$. The class of *primitive recursive functions relative to A* is the smallest class of partial recursive functions which contains:

- (1) the successor s ,
- (2) Z_k for every $k \in \mathbb{N}$,
- (3) I_k^i for every positive $k \in \mathbb{N}$ and $1 \leq i \leq k$,
- (4) the characteristic function $\mathbb{1}_A$ of A ,

and is closed under composition and primitive recursion.

Definition 7.2. Let R be a k -ary relation for some $k \in \mathbb{N}$ and let A be a subset of $\mathbb{N}$. Suppose that the characteristic function of R is primitive recursive relative to A . Then R is *primitive recursive relative to A* .

Remark 7.3. Let A be a subset of $\mathbb{N}$. Note that all results proved in previous sections for primitive recursive functions and relations hold for primitive recursive functions and relations relative to A .

Remark 7.4. Observe that primitive recursive functions coincide with the class of primitive recursive functions relative to $\emptyset$.

We introduce a class of functions, which naturally extends the class of primitive recursive functions. It was defined by Herbrand and Gödel in the early 1930s after publication of [?]. After that it was further clarified by Kleene.

Definition 7.5. Let A be a subset of $\mathbb{N}$. The class of *partial recursive functions relative to A* is the smallest class of partial functions that contains:

- (1) the successor s ,
- (2) Z_k for every $k \in \mathbb{N}$,
- (3) I_k^i for every positive $k \in \mathbb{N}$ and $1 \leq i \leq k$,
- (4) the characteristic function $\mathbb{1}_A$ of A ,

and is closed under composition, primitive recursion and unbounded minimization.

Definition 7.6. Let A be a subset of $\mathbb{N}$. A function is *recursive relative to A* if it is partial recursive relative to A and total.

8. LOOP-WHILE PROGRAMS

In this section we describe very basic programming languages. We fix a subset $A \subseteq \mathbb{N}$.

Definition 8.1. The *Loop-while language with oracle for A* consists of the following elements.

- Countable collection of variables

$$x_0, x_1, x_2, \dots$$

Each variable is able to store a number from $\mathbb{N}$.

- Class of commands defined by induction on their complexity.

- (1) $x_n := 0$ for $n \in \mathbb{N}$: This command sets the value of x_n to be zero.

- (2) $x_n := x_n + 1$ for $n \in \mathbb{N}$: This command increments the value of x_n .
- (3) $x_n := x_k$ for $n, k \in \mathbb{N}$: This command sets the value of x_n to be x_k .
- (4) If Q is a finite sequence of commands and $n \in \mathbb{N}$, then

$$\begin{array}{c} \text{loop } x_n \\ Q \\ \text{endloop} \end{array}$$

is a command.

- (5) If Q is a finite sequence of commands and $n \in \mathbb{N}$, then

$$\begin{array}{c} \text{while } x_n \\ Q \\ \text{endwhile} \end{array}$$

is a command.

- (6) An oracle command

$$x_n := \mathbb{1}_A(x_n)$$

for each $n \in \mathbb{N}$. This command sets the value of x_n to be $\mathbb{1}_A(x_n)$.

Definition 8.2. A *Loop language with oracle for A* is obtained from Loop-while language with oracle for A by removing while command.

Definition 8.3. A *Loop-while program with oracle for A* is a finite sequence of Loop-while commands with oracle for A .

Definition 8.4. A *Loop program with oracle for A* is a finite sequence of Loop commands with oracle for A .

Remark 8.5. Each Loop program with oracle for A is a Loop-while program with oracle for A .

Definition 8.6. A *state* of variables consists of configuration in which all but finitely many variables store zeros.

Definition 8.7. Let $\mathcal{P}$ be a Loop-while program with oracle for A . Suppose that it starts in some initial state. Then *execution of $\mathcal{P}$ starting from this initial state* follows iterative procedure described below:

- (1) At the beginning of each iteration the first command of the program is read.
- (2) Next the execution depends on the form of the command.

If the command is one of

$$x_n := 0, x_n := x_n + 1, x_n := x_k$$

for some $n, k \in \mathbb{N}$, then the command is executed and it is removed from the program.

If the first command has the form

$$\begin{array}{c} \text{loop } x_n \\ Q \\ \text{endloop} \end{array}$$

for some Q and $n \in \mathbb{N}$, then the value stored in x_n is retrieved. Suppose that it is $l \in \mathbb{N}$. Then the command is replaced in the program by concatenation of l copies of the subprogram Q .

If the first command has the form

$$\begin{array}{c} \text{while } x_n \\ \quad Q \\ \text{endwhile} \end{array}$$

for some Q and $n \in \mathbb{N}$, then the value stored in x_n is retrieved. If the value is nonzero, then the Q is executed. If the value is zero, then the command is removed from the program.

If the command is

$$x_n := \mathbb{1}_A(x_n)$$

for some $n \in \mathbb{N}$, then the command is executed and it is removed from the program.

Definition 8.8. A Loop-while program with oracle for A *terminates* if at some point during its execution its sequence of commands becomes empty.

Remark 8.9. Note that for every Loop program with oracle for A and every initial state of the execution the program starting from this initial state terminates. Indeed, the number of loop commands is decreasing during the execution and if there are no loop commands, then the program length decreases.

Definition 8.10. Let $\vec{n} \in \mathbb{N}^k$ for some $k \in \mathbb{N}$ and let $x_{r_1}, \dots, x_{r_k}$ be variables. Consider a state of variables defined by condition that x_{r_i} stores n_i for $i \in \{1, \dots, k\}$ and potentially some other variables store nonzero numbers. Then the state *stores $\vec{n}$ in variables $x_{r_1}, \dots, x_{r_k}$* .

Definition 8.11. Let $\mathcal{P}$ be a Loop-while program with oracle for A and let f be a k -ary partial function for some $k \in \mathbb{N}$. Suppose that the following assertions hold.

- (1) If $\vec{n} \in \mathbb{N}^k$ is any k -tuple in the complement of the domain of f , then executing $\mathcal{P}$ starting from the state with $\vec{n}$ stored in variables $x_1, \dots, x_k$ and with all other variables storing zero results in the program running indefinitely.
- (2) If $\vec{n} \in \mathbb{N}^k$ is any k -tuple in the domain of f , then executing $\mathcal{P}$ starting from the state with $\vec{n}$ stored in variables $x_1, \dots, x_k$ and with all other variables storing zero terminates and results in $f(\vec{n})$ stored in variable 0 upon termination.

Then f is *Loop-while computable relative to A* and $\mathcal{P}$ *computes f* .

Definition 8.12. A k -ary function f for some $k \in \mathbb{N}$ is *Loop computable relative to A* if there exists a Loop program $\mathcal{P}$ with oracle for A which computes f .

Remark 8.13. The Loop-while language relative to $\emptyset$ is referred to simply as the Loop-while language. Similarly the Loop language relative to $\emptyset$ is referred to as the Loop language. Note that these languages can be defined by removing oracle commands. We also refer to functions computable by Loop-while programs and Loop programs as Loop-while and Loop computable functions.

We prove now that some very basic functions are Loop computable.

Fact 8.14. Then zero constant k -ary function is Loop computable for every $k \in \mathbb{N}$.

Proof. Indeed, it is computed by the empty program. □

Fact 8.15. The successor function is Loop computable.

Proof. Indeed, the Loop program

$$x_1 := x_1 + 1$$

$$x_0 := x_1$$

computes the successor function. □

Fact 8.16. *The function I_n^k is Loop computable for $k, n \in \mathbb{N}$.*

Proof. Indeed, the Loop program

$$x_0 := x_k$$

computes I_n^k . □

Fact 8.17. *The function $\mathbb{1}_A$ is Loop computable relative to A .*

Proof. Indeed, the Loop program with oracle for A

$$x_0 := x_1$$

$$x_0 := \mathbb{1}_A(x_0)$$

computes $\mathbb{1}_A$. □

These are relatively simple examples. To demonstrate that the class of Loop-while computable functions relative to A includes more complex functions, we introduce the generalized notion of function computation.

Definition 8.18. Let $\mathcal{P}$ be a Loop-while program with oracle for A and let f be a k -ary partial function for some number $k \in \mathbb{N}$. Let $x_{r_1}, \dots, x_{r_k}, x_r$ be distinct variables and let l be a number. Suppose that the following assertions hold.

- (1) If $\vec{n} \in \mathbb{N}^k$ is in the complement of the domain of f , then the execution of $\mathcal{P}$ starting from the state with $\vec{n}$ in variables $x_{r_1}, \dots, x_{r_k}$ runs indefinitely.
- (2) If $\vec{n} \in \mathbb{N}^k$ is any k -tuple in the domain of f , then execution of $\mathcal{P}$ starting from the state with $\vec{n}$ in variables $x_{r_1}, \dots, x_{r_k}$ terminates, preserves initial contents of variables $x_0, \dots, x_{l-1}$ (excluding x_r if $r < l$) and stores $f(\vec{n})$ in x_r upon termination.

Then $\mathcal{P}$ computes f from $x_{r_1}, \dots, x_{r_k}$ to x_r preserving the first l variables.

Now we prove very useful result.

Proposition 8.19. *Let f be a k -ary Loop-while computable function relative to A for some $k \in \mathbb{N}$. Then for every distinct $x_{r_1}, \dots, x_{r_k}, x_r$ variables and every number l there exists a Loop-while program with oracle for A that computes f from $x_{r_1}, \dots, x_{r_k}$ to x_r preserving the first l variables.*

Moreover, if f is Loop computable relative to A , then there exists Loop program with oracle for A that computes f from $x_{r_1}, \dots, x_{r_k}$ to x_r preserving the first l variables.

Proof. Suppose that $\mathcal{P}$ is a Loop-while program with oracle for A that computes f . Our goal is to construct a Loop-while program $\mathcal{Q}$ with oracle for A that computes f from $x_{r_1}, \dots, x_{r_k}$ to x_r and preserves the first l variables.

Note that $\mathcal{P}$ is by definition a finite sequence of commands. Suppose that $M \in \mathbb{N}$ is greater than $r_1, \dots, r_k, r, l$ and indices of all variables occurring among commands of $\mathcal{P}$.

Having defined these numbers we construct $\mathcal{Q}$. We proceed in stages. Each stage is a Loop-while subprogram with oracle for A and $\mathcal{Q}$ is the concatenation of these subprograms.

- (1) $x_{M+i} := x_i$ for $i = 0, \dots, M - 1$.

(2) $x_i := 0$ for $i = 0, \dots, M - 1$.

(3) $x_i := x_{M+r_i}$ for $i = 1, \dots, k$.

(4) $\mathcal{P}$

(5) $x_r := x_0$

(6)

$$\begin{cases} x_i := x_{M+i} \text{ for } i = 0, \dots, l - 1 & \text{if } r \geq l \\ x_i := x_{M+i} \text{ for } i = 0, \dots, l - 1 \text{ with } r \text{ excluded} & \text{if } r < l \end{cases}$$

We now analyze $\mathcal{Q}$ and show that it satisfies the statement. Fix $\vec{n} \in \mathbb{N}^k$. Assume that $\mathcal{Q}$ is initialized with $\vec{n}$ in variables $x_{r_1}, \dots, x_{r_k}$. After (1), (2) and (3) all variables $x_0, \dots, x_{M-1}$ are zero except that $\vec{n}$ is stored in variables $x_1, \dots, x_k$. Next (4) is executing $\mathcal{P}$. We have two options.

- Assume that $\mathcal{P}$ runs indefinitely when initialized with $\vec{n}$ in variables $x_1, \dots, x_k$ and all other variables storing zeros. By the definition of M and considering that variables $x_0, \dots, x_{M-1}$ store zeros except for $x_1, \dots, x_k$ which store $\vec{n}$ we infer that (4) runs indefinitely.
- Assume that $\mathcal{P}$ terminates when initialized with $\vec{n}$ in variables $x_1, \dots, x_k$ and all other variables storing zero. By the definition of M and considering that variables $x_0, \dots, x_{M-1}$ store zeros except for $x_1, \dots, x_k$ which store $\vec{n}$ we infer that (4) terminates and results in $f(\vec{n})$ in x_0 . Finally (5) recovers the initial contents of the first l variables except for r if $r < l$.

This completes the proof of the main statement.

Note that if $\mathcal{P}$ is a Loop program with oracle for A , then also $\mathcal{Q}$ is a Loop program with oracle for A . This proves the additional statement. $\square$

Proposition 8.20. *Classes of Loop computable functions relative to A and Loop-while computable functions relative to A are closed under composition.*

Proof. Fix $k, n \in \mathbb{N}$. Let g be a k -ary Loop-while computable function relative to A and let $f_1, \dots, f_k$ be n -ary Loop-while computable functions relative to A . Let h be the composition of g with $f_1, \dots, f_k$. Our goal is to prove that h is Loop-while computable relative to A .

For every $i = 1, \dots, k$ let $\mathcal{P}_i$ be a Loop-while program with oracle for A that computes f_i from

$$\underbrace{x_1, \dots, x_n}_{\text{consecutive numbers}}$$

to x_{n+i} preserving the first $n + k + 1$ variables.

Next let $\mathcal{P}$ be a Loop-while program with oracle for A that computes g from

$$\underbrace{x_{n+1}, \dots, x_{n+k}}_{\text{consecutive numbers}}$$

to x_0 .

These programs exist according to Proposition 8.19.

Consider now the following Loop-while program with oracle for A .

(1) $\mathcal{P}_1, \dots, \mathcal{P}_k$

(2) $\mathcal{P}$

Suppose that the Loop-while program relative A is initialized with $\vec{m} \in \mathbb{N}^n$ stored in variables $x_1, \dots, x_n$. We need to distinguish three cases.

- All $f_1(\vec{m}), \dots, f_k(\vec{m})$ and $g(f_1(\vec{m}), \dots, f_k(\vec{m}))$ are defined. Then **(1)** terminates with $f_i(\vec{m})$ stored in variable x_{n+i} for each $i = 1, \dots, k$. Next **(2)** is executed, which also terminates and results in

$$h(\vec{m}) = g(f_1(\vec{m}), \dots, f_k(\vec{m}))$$

stored in x_0 .

- All $f_1(\vec{m}), \dots, f_k(\vec{m})$ are defined but $g(f_1(\vec{m}), \dots, f_k(\vec{m}))$ is undefined. Then **(1)** terminates and upon termination $f_i(\vec{m})$ is stored in variable x_{n+i} for each $i = 1, \dots, k$. Next **(2)** is executed, which runs indefinitely.
- At least one of $f_1(\vec{m}), \dots, f_k(\vec{m})$ is undefined. Then **(1)** runs indefinitely.

This completes the proof that h is a Loop-while computable relative to A .

Next if g and $f_1, \dots, f_k$ are Loop computable relative A , then we may pick $\mathcal{P}$ and $\mathcal{P}_1, \dots, \mathcal{P}_k$ to be Loop programs with oracle for A . Then also the program described above that computes h is a Loop program with oracle for A . Hence h is Loop computable relative to A . $\square$

Proposition 8.21. *Classes of Loop computable functions relative to A and Loop-while computable relative to A functions are closed under primitive recursion.*

Proof. Fix $k \in \mathbb{N}$. Let f be a k -ary Loop-while computable function relative to A and let g be a $(k+2)$ -ary Loop-while computable function relative to A . Suppose that h is obtained by primitive recursion from g and f . Our goal is to construct a Loop-while program with oracle for A which computes h .

Let $\mathcal{P}$ be a Loop-while program with oracle for A that computes f from $x_1, \dots, x_k$ to x_{k+3} by preserving the first $k+4$ variables. Similarly let $\mathcal{Q}$ be a Loop-while program with oracle for A that computes g from $\underbrace{x_1, \dots, x_k}_{\text{consecutive variables}}, x_{k+2}, x_{k+3}$ to variable x_{k+4} while preserving the first $k+4$ variables. Both these programs exist according to Proposition 8.19.

Consider now the following Loop-while program with oracle for A .

- (1) $\mathcal{P}$
- (2) loop x_{k+1}
 - $\mathcal{Q}$
 - $x_{k+3} := x_{k+4}$
 - $x_{k+2} := x_{k+2} + 1$
- endloop
- (3) $x_0 := x_{k+3}$

We now analyze this program and show that it satisfies the statement.

First we analyze the inner subprogram of the loop command **(2)**. Suppose that $\vec{n} \in \mathbb{N}^k$ is stored in variables $x_1, \dots, x_k$, the number $t \in \mathbb{N}$ in variable x_{k+2} and $h(\vec{n}, t)$ in variable x_{k+3} . We distinguish three cases.

- Assume that $h(\vec{n}, t+1)$ is defined. Then $\mathcal{Q}$ terminates and results in $h(\vec{n}, t+1)$ stored in x_{k+4} . Next $h(\vec{n}, t+1)$ is copied to x_{k+3} and x_{k+2} is incremented to $t+1$.
- Assume that $h(\vec{n}, t+1)$ is undefined. Then $\mathcal{Q}$ runs indefinitely.

Now fix $\vec{n} \in \mathbb{N}^k$ and $m \in \mathbb{N}$. Assume that the program described by **(1)**, **(2)** and **(3)** is initialized with $\vec{n}$ in variables $x_1, \dots, x_k$ and with m in variable x_{k+1} . We now execute the program. If f is not defined for $\vec{n}$, then **(1)** runs indefinitely and $h(\vec{n}, 0)$ is undefined. Thus we may assume that f is defined for $\vec{n}$, then **(1)** terminates and $h(\vec{n}, 0) = f(\vec{n})$ is stored in variable $k+3$. Moreover, the

first $k + 4$ variables are preserved. From our analysis of the subprogram consisting of the code block inside loop **(2)** we infer the following. Either $h(\vec{n}, m)$ is undefined and then $\mathcal{Q}$ inside **(2)** runs indefinitely or $h(\vec{n}, m)$ is defined and then the execution of **(2)** is basically execution of its inner subprogram m -times. This terminates and results in $h(\vec{n}, m)$ stored in variable $k + 3$. In the latter case $h(\vec{n}, m)$ is copied to x_0 by **(3)** and the whole program terminates. This proves that h is Loop-while computable relative to A .

Moreover, if g and f are Loop computable relative to A , then we may pick $\mathcal{P}$ and $\mathcal{Q}$ to be Loop programs with oracle for A . In that case the program consisting of **(1)**, **(2)** and **(3)** is a Loop program with oracle for A . Hence h is Loop computable relative to A . $\square$

Proposition 8.22. *The class of Loop-while computable functions relative to A is closed under unbounded minimization.*

Proof. Fix $k \in \mathbb{N}$. Let g be a $(k + 1)$ -ary partial Loop-while computable function relative to A . Suppose that f is the result of unbounded minimization of g . Our goal is to construct a Loop-while program with oracle for A which computes f .

Let $\mathcal{Q}$ be a Loop-while program with oracle for A that computes g from variables $x_1, \dots, x_k, x_{k+1}$ to variable x_{k+2} by preserving the first $k + 2$ variables. This program exists according to Proposition 8.19.

Consider now the following Loop-while program with oracle for A .

(1) $\mathcal{Q}$

(2) *while* x_{k+2}

$x_0 := x_0 + 1$

$x_{k+1} := x_{k+1} + 1$

$\mathcal{Q}$

endwhile

We now analyze this program and show that it satisfies the statement.

Suppose that $\vec{n} \in \mathbb{N}^k$ is stored in variables $x_1, \dots, x_k$ and all other variables store zeros. First note that if $g(\vec{n}, 0)$ is defined then after executing **(1)** contents of variables $x_0, x_1, \dots, x_k, x_{k+1}$ are preserved and x_{k+2} stores $g(\vec{n}, 0)$. Otherwise **(1)** runs indefinitely.

Next we assume that $\vec{n} \in \mathbb{N}^k$ is stored in variables $x_1, \dots, x_k$, variables x_0 and x_{k+1} store $t \in \mathbb{N}$, variable x_{k+2} stores $g(\vec{n}, t)$. We execute **(2)**. We distinguish three cases.

- Assume that $g(\vec{n}, t) \neq 0$ and $g(\vec{n}, t + 1)$ is defined. Then we execute the inner subprogram of **(2)**. First we increment x_0, x_{k+1} . Now there variables store $t + 1$. Next execute $\mathcal{Q}$. Since $g(\vec{n}, t + 1)$ is defined, $\mathcal{Q}$ terminates preserving the contents of the first $k + 1$ variables and stores $g(\vec{n}, t + 1)$ in x_{k+2} . This results in the following state:

$\vec{n}$ remains in variables $x_1, \dots, x_k$

$t + 1$ is stored in x_0 and x_{k+1}

x_{k+2} stores $g(\vec{n}, t + 1)$.

- Assume that $g(\vec{n}, t) \neq 0$ and $g(\vec{n}, t + 1)$ is undefined. Then we execute the inner subprogram of **(2)**. First we increment both x_0 and x_{k+1} . Next we execute $\mathcal{Q}$. Since $g(\vec{n}, t + 1)$ is undefined, $\mathcal{Q}$ runs indefinitely.
- Assume that $g(\vec{n}, t) = 0$. Then the program terminates with t in x_0 .

Suppose that the program is initialized with $\vec{n} \in \mathbb{N}^k$ in variables $x_1, \dots, x_k$. We use the analysis above to reason about the program's behavior.

If $f(\vec{n})$ is defined, then the program terminates and stores $f(\vec{n})$ in x_0 upon termination.

Assume now that $f(\vec{n})$ is undefined. Then $t \mapsto g(\vec{n}, t)$ is defined for some initial interval of $\mathbb{N}$, but its range does not include zero. In the analysis above shows that the program runs indefinitely. $\square$

Corollary 8.23. *The following assertions hold.*

- (1) *Each function partial recursive relative to A is Loop-while computable relative to A .*
- (2) *Each function primitive recursive relative to A is Loop computable relative to A .*

Proof. By Fact 8.14, Fact 8.15 and Fact 8.16 all basic functions are Loop computable and hence also Loop-while computable. According to Fact 8.17 also 1_A is Loop computable relative to A . Proposition 8.20, Proposition 8.21 and Proposition 8.22 imply that partial recursive functions relative to A are Loop-while computable relative to A . These results also prove that primitive recursive functions relative to A are Loop computable relative to A . $\square$

9. REGISTER MACHINES

We now define an abstract computing machine. A clear and early exposition of the idea of such a machine is presented in the paper [?]. We fix a subset $A \subseteq \mathbb{N}$.

Definition 9.1. *A register machine with oracle for A consists of the following components.*

- A set of *registers* numbered by elements of $\mathbb{N}$. Each register can store a number in $\mathbb{N}$. If $r \in \mathbb{N}$ is a register, then we denote by $c_r \in \mathbb{N}$ its content.
- A *program counter* capable of storing a number in $\mathbb{N}$. We denote its content by pc .
- Four types of *instructions*.

INC r for $r \in \mathbb{N}$: This instruction increments c_r and pc .

DEC r for $r \in \mathbb{N}$:

If $c_r > 0$, then the instruction decrements c_r and increments pc by 2.

If $c_r = 0$, then the instruction increments pc .

JUMP q for $q \in \mathbb{Z}$:

If $q \geq 0$, then the instruction increments pc by q .

If $q < 0$ and $|q| \leq pc$, then the instruction decrements pc by $|q|$.

If $q < 0$ and $pc < |q|$, then the instruction causes the machine to halt.

- Special *oracle instructions*.

ORACLE _{A} r for $r \in \mathbb{N}$:

If $c_r \notin A$, then the instruction increments pc .

If $c_r \in A$, then the instruction increments pc by 2.

Remark 9.2. The reader familiar with modern computers will recognize that a register machine with oracle for A is a mathematical model of a Harvard architecture central processing unit with random-access memory and equipped with a blackbox verifying if content of some register is in A .

Definition 9.3. A finite sequence of instructions is a *register-machine program with oracle for A* .

If $\mathcal{P}$ is a register-machine program with oracle for A , then we denote by $\text{lh}(\mathcal{P})$ the number of instructions in $\mathcal{P}$. Moreover, if $0 \leq i < \text{lh}(\mathcal{P})$, then $\mathcal{P}[i]$ denotes i -th instruction in $\mathcal{P}$.

Definition 9.4. A *state* of a register machine with oracle for A consists of all but finitely many registers storing zeros. Moreover, if the program counter is set to zero, then the state is *initial*.

Definition 9.5. Let $\mathcal{P}$ be a register-machine program with oracle for A . Suppose that a register machine starts in some initial state. Then *execution of $\mathcal{P}$ starting from this initial state* follows iterative procedure described below:

(1) At the beginning of each iteration the machine reads the content of the program counter.

(2) Next the execution depends on the content of program counter.

If the content of program counter is smaller than $\text{lh}(\mathcal{P})$, then the machine modifies contents of its registers and the program counter according to pc -th instruction in $\mathcal{P}$. Note that execution of this instruction may cause the machine to halt. If the instruction does not cause the machine to halt, then the next iteration begins.

If the content of program counter is greater or equal to the length of $\mathcal{P}$, then the machine halts.

Remark 9.6. Note that for every register-machine program with oracle for A and every initial state of registers execution of the program starting from this initial state may either continue indefinitely or halt at some iteration.

Definition 9.7. Let $\mathcal{P}$ be a register-machine program with oracle for A and suppose that a register-machine with oracle for A with some initial state is given. Assume that $\mathcal{P}$ is executed on the machine. If the machine halts with program counter equal to $\text{lh}(\mathcal{P})$, then the execution *terminates*.

We now provide examples of register-machine programs with oracle for A , which we use later as subprograms of larger programs.

Example 9.8. We define $\text{CLEAR } r$ for $r \in \mathbb{N}$ as the following program.

```
DEC  $r$ 
JUMP 2
JUMP  $-2$ 
```

Note that the effect of executing this program on a register machine is replacing the content of register r by zero.

Example 9.9. We define $\text{MOV } r, s$ for distinct $r, s \in \mathbb{N}$ as the following program.

```
CLEAR  $s$ 
DEC  $r$ 
JUMP 3
INC  $s$ 
JMP  $-3$ 
```

Note that the effect of executing this program on a register machine is twofold. First it replaces the content of register s by the initial content of register r . Second it replaces the initial content of r by zero.

Example 9.10. We define $\text{COPY } r, s, t$ for distinct $r, s, t \in \mathbb{N}$ as the following program.

```

CLEAR  $t$ 
CLEAR  $s$ 
DEC  $r$ 
JUMP 4
INC  $t$ 
INC  $s$ 
JUMP  $-4$ 
MOV  $t, r$ 

```

Note that the effect of executing this program on a register machine is threefold. First it replaces the content of register s by the initial content of register r . It preserves the initial content of register r . Finally it replaces the content of register t by zero.

We now show that every Loop-while program with oracle for A can be compiled to register-machine program with oracle for A . We fix a register machine with oracle for A . For each $r \in \mathbb{N}$ we identify the content of register r with the value of variable x_r . Under this identification states of Loop-while program with oracle for A correspond to states of the register machine with oracle for A .

Theorem 9.11. *There exists a function Γ that maps Loop-while programs with oracle for A to register-machine programs with oracle for A such that the following assertions hold.*

- (1) *Let $\mathcal{P}$ be a Loop-while program with oracle for A . Suppose that executions of $\mathcal{P}$ and $\Gamma(\mathcal{P})$ are started in the same state. If $\mathcal{P}$ runs indefinitely, then $\Gamma(\mathcal{P})$ runs indefinitely.*
- (2) *Let $\mathcal{P}$ be a Loop-while program with oracle for A . Suppose that executions of $\mathcal{P}$ and $\Gamma(\mathcal{P})$ are started in the same state. If the execution of $\mathcal{P}$ terminates, then the execution of $\Gamma(\mathcal{P})$ terminates and they both terminate in the same state.*

Proof. Recall that we identify state of variables with register machine states as explained above.

We fix $M \in \mathbb{N}$. Next we define a partially defined function Γ_M that maps Loop-while programs with oracle for A to register-machine programs with oracle for A . We require that Γ_M satisfies the following assertions.

Let $\mathcal{P}$ be a Loop-while program with oracle for A . Suppose that executions of $\mathcal{P}$ and $\Gamma_M(\mathcal{P})$ are started in the same state. If $\mathcal{P}$ runs indefinitely, then $\Gamma_M(\mathcal{P})$ runs indefinitely.

Let $\mathcal{P}$ be a Loop-while program with oracle for A . Suppose that executions of $\mathcal{P}$ and $\Gamma_M(\mathcal{P})$ are started in the same state. If the execution of $\mathcal{P}$ terminates, then the execution of $\Gamma_M(\mathcal{P})$ terminates and they both terminate in the same state.

Definition of Γ_M is by induction on complexity of Loop-while program with oracle for A .

First we define Γ_M for programs consisting of single basic commands. We set

$$\Gamma_M(x_n := 0) = \text{CLEAR } n, \Gamma_M(x_n := x_n + 1) = \text{INC } n$$

Note that this preserves input-output mapping: if the input state for one of the commands is S and the output state is R , then the corresponding register-machine program also maps S to R . Next we set

$$\Gamma_M(x_n := x_k) = \text{COPY } n, k, M$$

Here we assume that $n, k \in \mathbb{N}$ and $n, k < M$. Note that this also preserves input-output mapping under the assumption that in the input state the value stored in x_M is zero. Indeed, $\text{COPY } n, k, M$

clears register M upon termination. Here M serves as temporary helper register used during the execution of the copy operation.

We also set $\Gamma_M(x_n := \mathbb{1}_A(x_n))$ as follows:

```

ORACLEA  $n$ 
JUMP  $3 + \text{lh}(\text{CLEAR } n)$ 
CLEAR  $n$ 
INC  $n$ 
JUMP  $1 + \text{lh}(\text{CLEAR } n)$ 
CLEAR  $n$ 
    
```

Note that this also preserves input-output mapping.

Suppose now that $\mathcal{P}$ is a Loop-while program with oracle for A of the form

```

loop  $x_n$ 
   $\mathcal{Q}$ 
endloop
    
```

We assume by induction that $\Gamma_M(\mathcal{Q})$ is defined and satisfies the requirements. Then we set $\Gamma_M(\mathcal{P})$ as follows:

```

COPY  $n, M, M + 1$ 
DEC  $M$ 
JUMP  $\text{lh}(\Gamma_M(\mathcal{Q})) + 2$ 
 $\Gamma_M(\mathcal{Q})$ 
JUMP  $-\text{lh}(\Gamma_M(\mathcal{Q})) - 2$ 
CLEAR  $M$ 
    
```

This definition preserves the requirements assumption.

Suppose now that $\mathcal{P}$ is a Loop-while program with oracle for A of the form

```

while  $x_n$ 
   $\mathcal{Q}$ 
endwhile
    
```

We assume by induction that $\Gamma_M(\mathcal{Q})$ is defined and satisfy the requirements. Then we set $\Gamma_M(\mathcal{P})$ as follows:

```

DEC  $n$ 
JUMP  $\text{lh}(\Gamma_M(\mathcal{Q})) + 3$ 
INC  $n$ 
 $\Gamma_M(\mathcal{Q})$ 
JUMP  $-\text{lh}(\Gamma_M(\mathcal{Q})) - 3$ 
    
```

This definition preserves the requirements assumption.

Finally if $\mathcal{P}$ is a sequence of Loop-while commands with oracle for A , then we define $\Gamma_M(\mathcal{P})$ to be the concatenation of Γ_M applied to each command of $\mathcal{P}$.

Next fix a Loop-while program $\mathcal{P}$ with oracle for A . Let $M \in \mathbb{N}$ be larger than all indexes of variables occurring in $\mathcal{P}$. Then we define $\Gamma(\mathcal{P}) = \Gamma_M(\mathcal{P})$. By the constructions and induction above $\Gamma(\mathcal{P})$ satisfies properties (1) and (2). $\square$

Definition 9.12. Let $\vec{n} \in \mathbb{N}^k$ for some $k \in \mathbb{N}$ and let $r_1, \dots, r_k \in \mathbb{N}$ be registers. Consider an initial state of a register machine with oracle for A given by $c_{r_i} = n_i$ for $i \in \{1, \dots, k\}$ and potentially some other registers storing nonzero numbers. As usual assume that the machines program counter is set to zero. Then the machine is *initialized with $\vec{n}$ in registers $r_1, \dots, r_k$* .

Definition 9.13. Let $\mathcal{P}$ be a register-machine program with oracle for A and let f be a k -ary partial function for some number $k \in \mathbb{N}$. Suppose that the following assertions hold.

- (1) If $\vec{n} \in \mathbb{N}^k$ is any k -tuple in the complement of the domain of f , then executing $\mathcal{P}$ on a register machine initialized with $\vec{n}$ in registers $1, \dots, k$ and with all other registers storing zero results in the machine running indefinitely.
- (2) If $\vec{n} \in \mathbb{N}^k$ is any k -tuple in the domain of f , then executing $\mathcal{P}$ on a register machine initialized with $\vec{n}$ in registers $1, \dots, k$ and with all other registers storing zero terminates the execution and results in $f(\vec{n})$ stored in register 0 upon termination.

Then f is *register-machine computable relative to A* and $\mathcal{P}$ *computes f* .

Corollary 9.14. *Each Loop-while computable function relative to A is register-machine computable relative to A .*

Proof. Follows from Theorem 9.11. $\square$

Remark 9.15. The register machine relative to $\emptyset$ is referred to simply as register machine. Similarly register-machine programs relative to $\emptyset$ are referred to simply as register-machine programs. Note that in register-machine programs we can remove oracle instructions as they have no effect. We also refer to functions computable by register-machine programs as register-machine computable functions.

10. TURING-POST MACHINES

In this section we introduce the most rudimentary and finite model of computation developed independently by Turing ([?]) and Post ([?]) in 1936. Our definition follows the exposition by Davis in [?]. We fix a subset $A \subseteq \mathbb{N}$.

Definition 10.1. A *Turing-Post machine with oracle for A* consists of the following components.

- A *tape* of adjacent *cells* extending infinitely in both directions. Each cell stores one of two symbols 0, 1.
- A *head* pointing to some cell.
- The *program counter* storing a number in $\mathbb{N}$. Its content is denoted by pc .
- The set of *Turing-Post instructions*.

PRINT 0: writes symbol 0 in the cell to which head is pointing and increments pc .

PRINT 1: writes symbol 1 in the cell to which head is pointing and increments pc .

MOVE LEFT: moves head one cell left and increments pc .

MOVE RIGHT: moves head one cell right and increments pc .

IS ZERO: if the symbol in the cell to which head is pointing is 0, then increment pc by two, otherwise increment pc by one.

GOTO n : set pc to n for $n \in \mathbb{N}$.

- The *oracle instruction*.

ORACLE _{A} :

If the head points to symbol 0, then it increments pc .

Suppose that the head points to the beginning of the block of $c + 1$ consecutive 1s followed by 0 for some $c \in \mathbb{N}$. If $c \notin A$, then it increments pc , otherwise it increments pc by two.

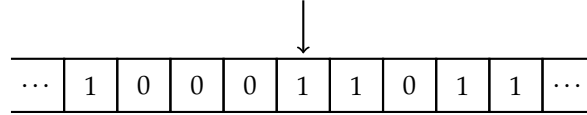


Figure 1 – The tape and head of Turing-Post machine with oracle for A

Definition 10.2. A *Turing-Post program with oracle for A* is a sequence of Turing-Post instructions and the oracle instruction.

If $\mathcal{P}$ is a Turing-Post program with oracle for A , then we denote by $\text{lh}(\mathcal{P})$ the number of instructions in $\mathcal{P}$. Moreover, if $0 \leq i < \text{lh}(\mathcal{P})$, then $\mathcal{P}[i]$ denotes i -th instruction in $\mathcal{P}$.

Definition 10.3. A *state* of a Turing-Post machine with oracle for A consists of:

- (1) the contents of all tape cells, with all but finitely many storing 0
- (2) the head position

A state is *initial* if $pc = 0$.

Definition 10.4. Let $\mathcal{P}$ be a Turing-Post program with oracle for A . Suppose that a Turing-Post machine with oracle for A starts in an initial state. Then *execution of $\mathcal{P}$ starting from this initial state* follows iterative procedure described below:

- (1) At the beginning of each iteration the machine reads the content of the program counter.
- (2) Next the execution depends on the content of the program counter.

If $pc < \text{lh}(\mathcal{P})$, then the machine modifies contents of its cells and the program counter according to pc -th instruction in $\mathcal{P}$.

If $\text{lh}(\mathcal{P}) \leq pc$, then the machine halts.

Definition 10.5. Let $\mathcal{P}$ be a Turing-Post program with oracle for A . Suppose $\mathcal{P}$ is executed on a Turing-Post machine with oracle for A starting from some initial state. If at some point of the execution the machine halts by attempting to fetch $\text{lh}(\mathcal{P})$ -th instruction, then the execution of $\mathcal{P}$ *terminates* when started in that initial state.

Remark 10.6. For additional flexibility we add two more Turing-Post instructions

JMP $+q$: increments pc by $q \in \mathbb{N}$

JMP $-q$: decrements pc by $q \in \mathbb{N}$; only valid if $q \leq pc$.

Note that these both instructions can be compiled into the standard GOTO instructions.

Note that Turing-Post machines and programs with oracle for A are extremely simple. Actually the whole point of introducing them is to show that a computation only requires very rudimentary and finite operations.

Our goal is to show that register-machine programs with oracle for A can be compiled into Turing-Post programs with oracle for A . To do this, we encode register states as tape states. Fix a state of register machine and assume that only the first $R \in \mathbb{N}$ registers contain nonzero values. Represent this state on the tape by R blocks of consecutive 1s with each pair of consecutive blocks separated by a single 0. The $(r+1)$ -th block from the left consists of $c_r + 1$ consecutive 1s. The head points to the first cell encoding register 0.

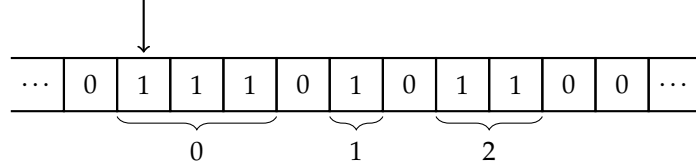


Figure 2 – The tape and head of Turing-Post machine with oracle for A representing the initial state of register machine with $c_0 = 2, c_1 = 0, c_2 = 2$ and all other registers storing zeros

Remark 10.7. Note that under this encoding different tape states may represent the same register machine state.

Now we define some helper subroutines for Turing-Post machines with oracle for A .

Example 10.8. We define subroutine NEXT REGISTER.

```

MOVE RIGHT
IS ZERO
JMP - 2
MOVE RIGHT

```

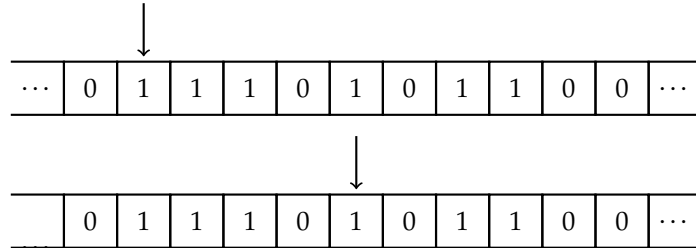


Figure 3 – Subroutine NEXTREGISTER moves the head to the first cell representing next register.

Example 10.9. We define subroutine PREVIOUS REGISTER.

```

MOVE LEFT
MOVE LEFT
MOVE LEFT
IS ZERO

```

JMP – 2

MOVE RIGHT

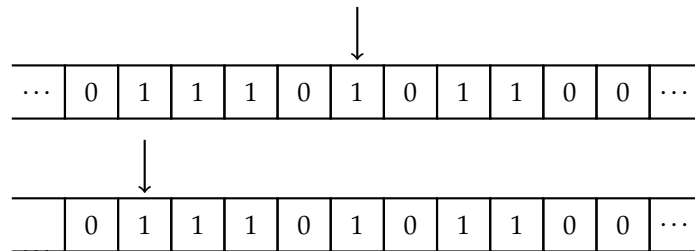


Figure 4 – Subroutine PREVIOUS REGISTER moves the head to the first cell representing the content of the previous register.

Example 10.10. We define subroutine SHIFT BLOCK OF 1s RIGHT.

PRINT 0

MOVE RIGHT

IS ZERO

JMP – 2

PRINT 1

MOVE RIGHT

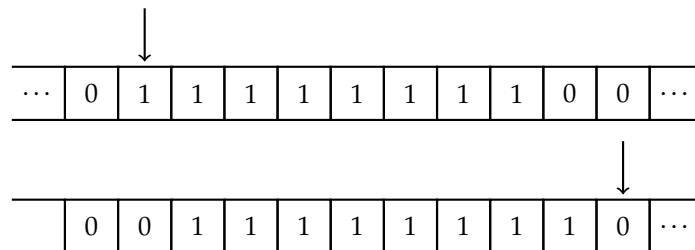


Figure 5 – Subroutine SHIFT BLOCK OF 1s RIGHT shifts the block of 1s right and moves the head to the right of the shifted block.

Example 10.11. We define subroutine SHIFT BLOCK OF 1s LEFT.

MOVE LEFT

PRINT 1

MOVE RIGHT

IS ZERO

JMP – 2

MOVE LEFT

PRINT 0

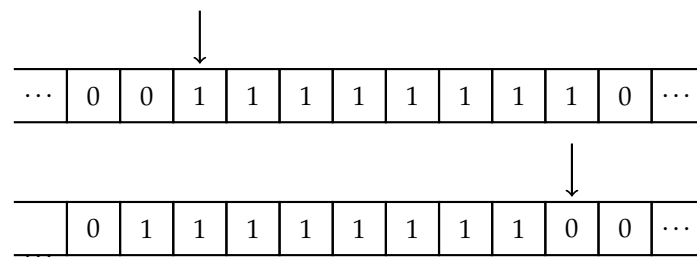


Figure 6 – Subroutine SHIFT BLOCK OF 1s LEFT shifts the block of 1s right and moves the head right to the shifted block.

Example 10.12. We define subroutine INCREASE BLOCK OF 1s.

MOVE RIGHT

IS ZERO

JMP – 2

PRINT 1

MOVE RIGHT

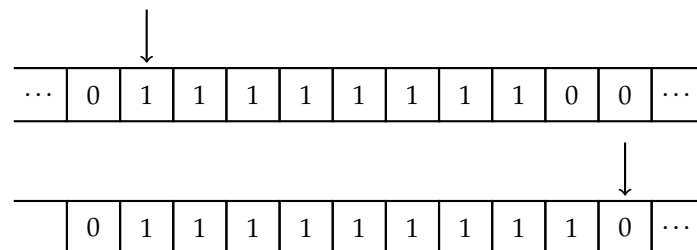


Figure 7 – Subroutine INCREASE BLOCK OF 1s increases the block of 1s by adding one cell on the right and moves the head to the right of the increased block.

Example 10.13. We define subroutine DECREASE BLOCK OF 1s.

MOVE RIGHT

IS ZERO

JMP – 2

MOVE LEFT

PRINT 0

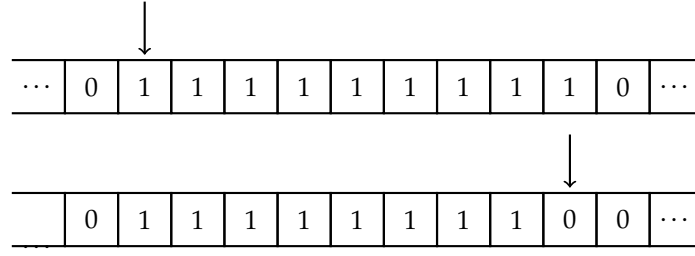


Figure 8 – Subroutine DECREASE BLOCK OF 1s decreases the block of 1s by adding one cell on the right and moves the head right to the decreased block.

Theorem 10.14. *There exists a function Γ that maps register-machine programs with oracle for A to Turing-Post programs with oracle for A such that the following assertions hold.*

- (1) *Let $\mathcal{P}$ be a register-machine program with oracle for A and let S be a state of a register machine with oracle for A . Suppose that executions of $\mathcal{P}$ and $\Gamma(\mathcal{P})$ are started in S . Then the sequence of consecutive states of $\mathcal{P}$ is a subsequence of the sequence of the consecutive states of $\Gamma(\mathcal{P})$.*
- (2) *Let $\mathcal{P}$ be a register-machine program with oracle for A and let S be a state of a register machine with oracle for A . Suppose that executions of $\mathcal{P}$ and $\Gamma(\mathcal{P})$ are started in S . If the execution of $\mathcal{P}$ terminates, then the execution of $\Gamma(\mathcal{P})$ terminates and they both terminate in the same state.*

Proof. We augment Turing-Post instructions by adding new syntactical constructs. For each number $n \in \mathbb{N}$ we introduce an entity SYMBOL_n which represents n . SYMBOL_n for $n \in \mathbb{N}$ has a twofold role.

- We replace instruction $\text{GO TO } n$ by

$$\text{GO TO SYMBOL}_{L_n}$$
 for $n \in \mathbb{N}$.
- SYMBOL_{L_n} can be placed in front of instructions.

We now define some extra Turing-Post subroutines.

We define subroutine $\text{MOVE TO REGISTER } r$ for $r \in \mathbb{N}$ by concatenating r -times the following block of instructions.

```
IS ZERO
JMP + 2
PRINT 1
NEXT REGISTER
```

and appending the block of instructions

```
IS ZERO
JMP + 2
PRINT 1
```

at the end. If the head points to the beginning of the block representing the first register, then the subroutine moves the head to the beginning of the block representing the content of register r . It creates any intermediate blocks if needed and the appended instructions ensure that the block representing register r is properly initialized.

We define subroutine $\text{MOVE TO FIRST REGISTER}$ as the following sequence of instructions.

PREVIOUS REGISTER
 IS ZERO
 $\text{JMP} - 1 - \text{lh}(\text{PREVIOUS REGISTER})$
 MOVE RIGHT
 MOVE RIGHT

If the head points to the beginning of block representing some register, then the subroutine moves the head to the beginning of the block representing the first register.

We define $I_{r,n}$ for $r, n \in \mathbb{N}$ by placing SYMBOL_n in front of the first instruction of the following subroutine.

- (1) MOVE TO REGISTER r
- (2) INCREASE BLOCK OF 1s
- (3) IS ZERO
- (4) $\text{JMP} + 2$
- (5) $\text{JMP} + 2 + \text{lh}(\text{SHIFT BLOCK OF 1s RIGHT})$
- (6) SHIFT BLOCK OF 1s RIGHT
- (7) $\text{JMP} - \text{lh}(\text{SHIFT BLOCK OF 1s RIGHT}) - 3$
- (8) MOVE RIGHT
- (9) MOVE TO FIRST REGISTER

We describe the effect of this subroutine on the tape of the machine. Suppose that the head points to the beginning of the block representing first register. Step (1) moves the head to the beginning of the block representing register r . Next (2) increases the block representing register r by appending single 1 at the end. Now (3)-(7) shift all blocks following the one representing register r one cell to the right. Finally (8) and (9) return the head back to the beginning of the block representing the first register.

We define $D_{r,n}$ for $r, n \in \mathbb{N}$ by placing SYMBOL_n in front of the first instruction in the following subroutine.

- (1) MOVE TO REGISTER r
- (2) MOVE RIGHT
- (3) IS ZERO
- (4) $\text{JMP} + 3 + \text{lh}(\text{MOVE TO FIRST REGISTER})$
- (5) MOVE RIGHT
- (6) MOVE TO FIRST REGISTER
- (7) GO TO SYMBOL_{n+1}
- (8) DECREASE BLOCK OF 1s
- (9) MOVE RIGHT
- (10) MOVE RIGHT
- (11) IS ZERO
- (12) $\text{JMP} + 2$
- (13) $\text{JMP} + 2 + \text{lh}(\text{SHIFT BLOCK OF 1s LEFT})$

- (14) SHIFT BLOCK OF 1s LEFT
- (15) $\text{JMP} - \text{lh}(\text{SHIFT BLOCK OF 1s LEFT}) - 5$
- (16) MOVE LEFT
- (17) MOVE TO FIRST REGISTER
- (18) GO TO SYMBOL_{n+2}

We describe the effect of this subroutine on the tape of the machine. Suppose that the head points to the beginning of the block representing first register. Step (1) moves the head to the beginning of the block representing register r . Next (2) and (3) check whether register r stores zero. We have two cases.

- If register r stores zero, then (5) and (6) move the head to the beginning of the block representing the first register. Next (7) jumps to SYMBOL_{n+1} .
- If register r stores a nonzero number, then (4) transfers the control to (8) which decreases the block representing register r by removing rightmost 1 and moves the head to the right of the decreased block. Next (9)-(15) shift all blocks following the block representing register r one cell to the left. Steps (16) and (17) return the head to the beginning of the block representing the first register. Finally (18) jumps to SYMBOL_{n+2} .

We define $O_{r,n}$ for $r, n \in \mathbb{N}$ by placing SYMBOL_n in front of the first instruction in the following subroutine.

- (1) MOVE TO REGISTER r
- (2) ORACLE_A
- (3) $\text{JMP} + \text{lh}(\text{MOVE TO FIRST REGISTER}) + 2$
- (4) MOVE TO FIRST REGISTER
- (5) GO TO SYMBOL_{n+1}
- (6) MOVE TO FIRST REGISTER
- (7) GO TO SYMBOL_{n+2}

We describe the effect of this subroutine on the tape of the machine. Suppose that the head points to the beginning of the block representing first register. Step (1) moves the head to the beginning of the block representing register r . Next (2) verifies if the content of register r is in A . We have two cases.

- If register r stores an element of A , then (3) jumps to (6). Next (6) moves the head to the beginning of the block representing the first register. Finally (7) jumps to SYMBOL_{n+2} .
- If register r does not store an element of A , then (4) moves the head to the beginning of the block representing the first register. Next (5) jumps to SYMBOL_{n+1} .

Now we are ready to define Γ . This is done by constructing an intermediate representation.

Suppose that $\mathcal{P}$ is a register-machine program with oracle for A .

- If n -th instruction of $\mathcal{P}$ is $\text{INC } r$ for some $r \in \mathbb{N}$, then we compile it to $I_{r,n}$.
- If n -th instruction of $\mathcal{P}$ is $\text{DEC } r$ for some $r \in \mathbb{N}$, then we compile it to $D_{r,n}$.
- If n -th instruction of $\mathcal{P}$ is $\text{JUMP } q$ for some $q \in \mathbb{Z}$, then we compile it to $\text{GO TO SYMBOL}_{n+q}$.
- If n -th instruction of $\mathcal{P}$ is $\text{ORACLE}_A r$ for some $r \in \mathbb{N}$, then we compile it to $O_{r,n}$.

The concatenation of the compiled instructions in the correct order is the intermediate representation $\tilde{\mathcal{P}}$ of $\mathcal{P}$.

Next for every SYMBOL_n for $n \in \mathbb{N}$ appearing in front of some instruction in $\tilde{\mathcal{P}}$.

- (1) Remove it from this instruction.
- (2) Replace all other occurrences of SYMBOL_n by the position of that instruction.

Perform this procedure for all SYMBOL_n occurring in front of instructions of $\tilde{\mathcal{P}}$. Next for every $\text{SYMBOL}_{\text{lh}(\mathcal{P})}$ appearing in $\tilde{\mathcal{P}}$ replace it by $\text{lh}(\tilde{\mathcal{P}})$. If any symbols remain as parts of go-to instructions after this, then replace them by $\text{lh}(\tilde{\mathcal{P}}) + 1$. This transformation produces $\Gamma(\mathcal{P})$.

According to descriptions of subroutines $I_{r,n}, D_{r,n}, O_{r,n}$ for $r, n \in \mathbb{N}$ and by straightforward observation that $\text{JUMP } q$ at n -th position in register-machine program is semantically equivalent to Turing-Post $\text{GO TO SYMBOL}_{n+q}$ for $n \in \mathbb{N}$ and $q \in \mathbb{Z}$, we deduce that $\Gamma(\mathcal{P})$ satisfies the assertions in the statement of the theorem. $\square$

Definition 10.15. Let $\vec{n} \in \mathbb{N}^k$ for some $k \in \mathbb{N}$. Consider a state of a Turing-Post machine with oracle for A consisting of k contiguous blocks of 1s separated by single 0s such that the i -th block from the left contains n_i ones for $1 \leq i \leq k$ and the head points to the beginning of the first block. Then the machine *stores* $\vec{n}$.

Definition 10.16. Let $\mathcal{P}$ be a Turing-Post program with oracle for A and let f be a k -ary partial function for some $k \in \mathbb{N}$. Suppose that the following assertions hold.

- (1) If $\vec{n} \in \mathbb{N}^k$ is any k -tuple in the complement of the domain of f , then executing $\mathcal{P}$ on a Turing-Post machine with oracle for A initialized with $\vec{n}$ stored on the tape runs indefinitely.
- (2) If $\vec{n} \in \mathbb{N}^k$ is any k -tuple in the domain of f , then executing $\mathcal{P}$ on a Turing-Post machine with oracle for A initialized with $\vec{n}$ stored on the tape terminates and results in the state in which the tape contains single contiguous block of $f(\vec{n})$ ones and the head points to the beginning of the block.

Then f is *Turing-Post computable relative to A* and $\mathcal{P}$ *computes f* .

Corollary 10.17. *Each register-machine computable function relative to A is Turing-Post computable relative to A .*

Proof. Let f be a k -ary register-machine computable function relative to A and let $\mathcal{P}$ be a register-machine program with oracle for A that computes f . Suppose that $M \in \mathbb{N}$ is greater than k and all registers occurring in instructions of $\mathcal{P}$. Define $\mathcal{Q}$ by appending to $\mathcal{P}$ instructions $\text{CLEAR } r$ for $0 < r < M$. Then $\mathcal{Q}$ computes f and clears all registers except register 0 upon termination.

There exists a Turing-Post program with oracle for A which when initialized with $\vec{n}$ stored on the tape for $\vec{n} \in \mathbb{N}^k$ terminates with the machine storing

$$(1, n_1 + 1, \dots, n_k + 1)$$

We leave the implementation of this program as an exercise for the reader and name this subroutine **PREPARE TAPE**.

Next there exists a Turing-Post program with oracle for A which starts with the machine storing the state of the register machine such that $c_r = 0$ for all positive $r \in \mathbb{N}$ and upon termination leaves a unique block of c_0 ones with the head pointing to the beginning of the block. The implementation of this program is also left as an exercise. We name this subroutine **CLEAN TAPE**.

Finally Turing-Post program with oracle for A

PREPARE TAPE

$\Gamma(Q)$

CLEAN TAPE

computes f , where Γ is defined in Theorem 10.14. $\square$

11. THE LANGUAGE OF ARITHMETIC

In this section we fix a subset $A \subseteq \mathbb{N}$.

Definition 11.1. The *language of arithmetic* is the first order language with identity whose signature consists of the following symbols:

$$\underbrace{0}_{\text{constant}}, \underbrace{<}_{\text{binary relation symbol}}, \underbrace{S}_{\text{unary function symbol}}, \underbrace{+, \cdot}_{\text{binary function symbols}}$$

We also denote this language with $\mathbf{Ar}$.

The intended model of $\mathbf{Ar}$ is $\mathfrak{N} = (\mathbb{N}, 0, s, <, +, \cdot)$ where symbols $0, <, +, \cdot$ have their usual meaning and s denotes the successor function i.e. $s(n) = n + 1$ for every $n \in \mathbb{N}$.

For every $n \in \mathbb{N}$ the term $\underbrace{S \cdot \dots \cdot S}_{n \text{ times}} 0$ of $\mathbf{Ar}$ is denoted by $\bar{n}$.

We also fix a bijection between the first countable ordinal and the collection of variables of $\mathbf{Ar}$. This induces a well-ordering on variables of $\mathbf{Ar}$.

Definition 11.2. The *language of arithmetic relative to A* is the extension of $\mathbf{Ar}$ by

$$\underbrace{r_A}_{\text{unary relation symbol}}$$

We denote this language by $\mathbf{Ar}_A$.

The intended model of $\mathbf{Ar}_A$ is the expansion of $\mathfrak{N}$ by interpreting r_A as $A \subseteq \mathbb{N}$. This model of $\mathbf{Ar}_A$ is denoted by $\mathfrak{N}_A$.

Definition 11.3. Let ϕ be a formula in $\mathbf{Ar}_A$ and let $x_1, \dots, x_k$ be all free variables of ϕ listed according to the fixed well-order on the variables of $\mathbf{Ar}_A$. Then $x_1, \dots, x_k$ is the *context* of ϕ .

Let ϕ be a formula of $\mathbf{Ar}_A$ and let $x_1, \dots, x_k$ be the context of ϕ . For $n_1, \dots, n_k \in \mathbb{N}$ we denote by $\phi(n_1, \dots, n_k)$ the sentence obtained from ϕ by substituting x_i by $\bar{n}_i$ for $i = 1, \dots, k$.

Definition 11.4. Let ϕ be a formula in $\mathbf{Ar}_A$ and let $x_1, \dots, x_k$ be the context of ϕ . Then we set

$$\phi^{\mathfrak{N}_A} = \{(n_1, \dots, n_k) \in \mathbb{N}^k \mid \phi(n_1, \dots, n_k) \text{ holds in } \mathfrak{N}\}$$

Then k -ary relation $\phi^{\mathfrak{N}} \subseteq \mathbb{N}^k$ is the *interpretation* of ϕ in $\mathfrak{N}_A$.

Definition 11.5. Let ϕ and ψ be formulas in $\mathbf{Ar}_A$. If $\phi^{\mathfrak{N}_A} = \psi^{\mathfrak{N}_A}$, then ϕ and ψ are *arithmetically equivalent*.

Definition 11.6. Let R be a k -ary relation and let ϕ be a formula of $\mathbf{Ar}_A$. Then R is *definable* by ϕ if $\phi^{\mathfrak{N}_A}$ coincides with R .

Remark 11.7. Let ϕ be a formula in $\mathbf{Ar}_A$ and let x, y be distinct variables. Then we introduce the following notation

$$\begin{aligned} x \leq y &\equiv (x = y) \vee (x < y) \\ \forall_{x < y} \phi &\equiv \forall_x ((y \leq x) \vee \phi), \exists_{x < y} \phi \equiv \exists_x ((x < y) \wedge \phi) \end{aligned}$$

$$\forall_{x \leq y} \phi \equiv \forall_x ((y < x) \vee \phi), \exists_{x \leq y} \phi \equiv \exists_x ((x \leq y) \wedge \phi)$$

12. RUDIMENTARY AND EXISTENTIAL RUDIMENTARY FORMULAS

We fix a subset $A \subseteq \mathbb{N}$. In this section we introduce important class of formulas of $\mathbf{Ar}_A$.

Definition 12.1. The class of *rudimentary formulas relative to A* is the smallest class of formulas in $\mathbf{Ar}_A$ such that the following conditions hold.

- (1) Every atomic formula is rudimentary relative to A .
- (2) If ϕ, ψ are rudimentary formulas relative to A , then

$$(\phi \wedge \psi), (\phi \vee \psi)$$

are rudimentary relative to A .

- (3) If ϕ is a rudimentary formula relative to A , then $(\neg \phi)$ is rudimentary relative to A .
- (4) if ϕ is a rudimentary formula relative to A and x, y are variables, then also

$$\forall_{x < y} \phi, \forall_{x \leq y} \phi, \exists_{x < y} \phi, \exists_{x \leq y} \phi$$

are rudimentary relative to A .

Definition 12.2. Let ϕ be a rudimentary formula relative to A and let x be a variable. Then $\exists_x \phi$ is a $\exists$ -rudimentary formula relative to A .

Proposition 12.3. The following assertions hold.

- (1) If ϕ is $\exists$ -rudimentary formula relative to A and x is variable, then $\exists_x \phi$ is arithmetically equivalent to $\exists$ -rudimentary formula relative to A .
- (2) If ϕ and ψ are $\exists$ -rudimentary formulas relative to A , then formulas $(\phi \wedge \psi)$ and $(\phi \vee \psi)$ are arithmetically equivalent to $\exists$ -rudimentary formulas relative to A .
- (3) If ϕ is $\exists$ -rudimentary formula relative to A and x, y are variables, then formulas

$$\forall_{x < y} \phi, \forall_{x \leq y} \phi, \exists_{x < y} \phi, \exists_{x \leq y} \phi$$

are arithmetically equivalent to $\exists$ -rudimentary formulas relative to A .

Proof. Assume that ϕ is $\exists$ -rudimentary relative to A . Then $\phi \equiv \exists_z \theta$ for some rudimentary formula θ relative to A and variable z . Pick variable t different from both x, z and all variables occurring in θ . Then $\exists_x \exists_z \theta$ is arithmetically equivalent to $\exists$ -rudimentary formula $\exists_t \exists_{x < t} \exists_{z < t} \theta$ relative to A . This proves (1).

Now suppose that ϕ and ψ are $\exists$ -rudimentary relative to A . Then there exist rudimentary formulas θ, η relative to A and variables x, y such that $\phi \equiv \exists_x \theta$, $\psi \equiv \exists_y \eta$. Now pick variable t different from x, y and all variables occurring in θ and η . Then

$$(\phi \wedge \psi) \equiv (\exists_x \theta \wedge \exists_y \eta)$$

is arithmetically equivalent to $\exists$ -rudimentary formula

$$\exists_t (\exists_{x < t} \theta \wedge \exists_{y < t} \eta)$$

relative to A . Similar argument works for $\vee$. This proves (2).

Assume that ϕ is $\exists$ -rudimentary relative to A . Then $\phi \equiv \exists_z \theta$ for some rudimentary formula θ and variable z . Pick variable t different from z, x, y and all variables occurring with θ . Then

$$\forall_{x < y} \exists_z \theta$$

is arithmetically equivalent to $\exists$ -rudimentary formula

$$\exists_t \forall_{x < y} \exists_{z < t} \theta$$

relative to A . Similar argument works with $\forall_{x \leq y} \phi$. Consider now $\exists_{x < y} \phi$. By definition this formula is

$$\exists_x ((x < y) \wedge \phi)$$

and hence it is arithmetically equivalent to $\exists$ -rudimentary relative to A by (1) and (2). Similar argument works with $\exists_{x \leq y} \phi$. This completes the proof of (3). $\square$

13. EXISTENTIAL RUDIMENTARY FUNCTIONS AND RELATIONS

We fix a subset $A \subseteq \mathbb{N}$.

Definition 13.1. Let R be a k -ary relation definable by $\exists$ -rudimentary formula relative to A . Then R is an $\exists$ -rudimentary relation relative to A .

Definition 13.2. Let f be a k -ary partial function such that Γ_f is $\exists$ -rudimentary relation relative to A . Then f is an $\exists$ -rudimentary function relative to A .

Remark 13.3. Rudimentary and $\exists$ -rudimentary formulas relative to $\emptyset$ are referred to simply as rudimentary and $\exists$ -rudimentary. Similarly $\exists$ -rudimentary relations and functions relative to $\emptyset$ are referred to as $\exists$ -rudimentary relations and functions.

Fact 13.4. The following functions are $\exists$ -rudimentary.

- (1) The zero k -ary function for every $k \in \mathbb{N}$.
- (2) I_k^i for every $k \in \mathbb{N}$ and every $1 \leq i \leq k$.
- (3) The successor, the addition and the multiplication.

Proof. The proof is left for the reader. $\square$

Fact 13.5. The function $\mathbb{1}_A$ is $\exists$ -rudimentary relative to A .

Proof. Indeed, the formula

$$(y = \bar{1} \wedge r_A x) \vee (y = \mathbf{0} \wedge (\neg r_A x))$$

is $\exists$ -rudimentary relative to A and defines $\mathbb{1}_A$. $\square$

Proposition 13.6. The class of $\exists$ -rudimentary functions relative to A is closed under composition.

Proof. Suppose that $f_1, \dots, f_k$ are l -ary $\exists$ -rudimentary functions relative to A and g is a k -ary $\exists$ -rudimentary function relative to A . Let $\phi_1, \dots, \phi_k$ be $\exists$ -rudimentary formulas relative to A defining $f_1, \dots, f_k$. By renaming free variables we may assume that the context of ϕ_i is $x_1, \dots, x_l, y_i$ for $i = 1, \dots, k$. Then

$$f_i(m_1, \dots, m_l) = m \Leftrightarrow \phi_i(m_1, \dots, m_l, m) \text{ holds in } \mathfrak{N}$$

for every $m_1, \dots, m_l, m \in \mathbb{N}$ and $i = 1, \dots, k$. Let ϕ be an $\exists$ -rudimentary formula relative to A defining g . By renaming free variables we may assume that the context of ϕ is $y_1, \dots, y_k, y$ and

$$g(n_1, \dots, n_k) = n \Leftrightarrow \phi(n_1, \dots, n_k, n) \text{ holds in } \mathfrak{N}$$

for all $n_1, \dots, n_k, n \in \mathbb{N}$. Then

$$\exists_{y_1} \exists_{y_2} \dots \exists_{y_k} (\phi_1 \wedge \dots \wedge \phi_k \wedge \phi)$$

is a formula of $\mathbf{Ar}_A$ which defines the composition of g with $f_1, \dots, f_k$. By Proposition 12.3 we infer that the formula above is arithmetically equivalent to $\exists$ -rudimentary formula relative to A . Thus the composition of g with $f_1, \dots, f_k$ is $\exists$ -rudimentary relative to A . This completes the proof. $\square$

Now we prove nontrivial result proved in [?].

Theorem 13.7 [Gödel]. *The class of $\exists$ -rudimentary functions relative to A is closed under primitive recursion.*

We first need to prove the following interesting result.

Lemma 13.7.1 [Beta function]. *There exists $\exists$ -rudimentary total 3-ary function β such that for every $n \in \mathbb{N}$ and every sequence $a_0, \dots, a_n \in \mathbb{N}$ there exist $a, b \in \mathbb{N}$ such that*

$$\beta(a, b, i) = a_i$$

for $0 \leq i \leq n$.

Proof of the lemma. Consider a rudimentary formula

$$\exists_t \left(\forall_{z \leq t} (z \cdot y < x) \wedge (x \leq S(t) \cdot y) \wedge (r + t \cdot y = x) \right)$$

in the context x, y, r . Then this formula defines a 2-ary function which maps a pair (n, m) to the remainder of division of n by m .

Now we define

$$\beta(n_1, n_2, n_3) = \text{rem}(n_1, (n_3 + 1) \cdot n_2 + 1)$$

Then β is 3-ary $\exists$ -rudimentary function by Fact 13.4 and Proposition 13.6.

Now pick $n \in \mathbb{N}$ and a sequence $a_0, \dots, a_n \in \mathbb{N}$. Suppose that

$$\max(a_0, \dots, a_n, n) \leq m$$

for some $m \in \mathbb{N}$. Then numbers

$$(i + 1) \cdot m! + 1$$

for $0 \leq i \leq n$ are pairwise coprime. According to Chinese remainder theorem there exists $a \in \mathbb{N}$ such that

$$\beta(a, m!, i) = \text{rem}(a, (i + 1) \cdot m! + 1) = a_i$$

for $0 \leq i \leq n$. This completes the proof. $\square$

Proof of the theorem. Suppose that g is $(k + 2)$ -ary $\exists$ -rudimentary function relative to A and f is k -ary $\exists$ -rudimentary function relative to A . Let ϕ be an $\exists$ -rudimentary formula that defines β from Lemma 13.7.1. Without loss of generality assume that the context of ϕ is z_1, z_2, z_3, z and

$$\beta(n_1, n_2, n_3) = n \Leftrightarrow \phi(n_1, n_2, n_3, n) \text{ holds in } \mathfrak{N}$$

for all $n_1, n_2, n_3, n \in \mathbb{N}$. Let θ be an $\exists$ -rudimentary formula relative to A with context $x_1, \dots, x_k, y, z, w$ such that

$$g(n_1, \dots, n_k, m, n) = l \Leftrightarrow \theta(n_1, \dots, n_k, m, n, l) \text{ holds in } \mathfrak{N}$$

for all $n_1, \dots, n_k, m, n, l \in \mathbb{N}$. Let ψ be an $\exists$ -rudimentary formula relative to A with context $x_1, \dots, x_k, u$ such that

$$f(n_1, \dots, n_k) = l \Leftrightarrow \psi(n_1, \dots, n_k, l) \text{ holds in } \mathfrak{N}$$

for all $n_1, \dots, n_k, l \in \mathbb{N}$. Moreover, without loss of generality we may assume that all variables

$$z_1, z_2, z_3, x_1, \dots, x_k, y, z, w, u$$

are distinct. Let t be a variable distinct from all these variables. Consider the formula

$$\exists_{z_1} \exists_{z_2} \left(\exists_u ([\phi]_{0,u}^{z_3, z} \wedge \psi) \wedge \forall_{t < y} \exists_w \exists_z ([\phi]_t^{z_3} \wedge [\theta]_t^y \wedge [\phi]_{S(t), w}^{z_3, z}) \wedge [\phi]_y^{z_3} \right)$$

Proposition 12.3 shows that the formula above is arithmetically equivalent to an $\exists$ -rudimentary formula relative to A . By Lemma 13.7.1 we infer that the formula defines $(k + 1)$ -ary function obtained by primitive recursion from g and f . $\square$

Corollary 13.8. *The following assertions hold.*

- (1) Each primitive recursive function relative to A is $\exists$ -rudimentary relative to A .
- (2) Each primitive recursive relation relative to A is $\exists$ -rudimentary relative to A .

Proof. The assertion (1) follows immediately from Fact 13.4, Fact 13.5, Proposition 13.6 and Theorem 13.7.

Let R be a k -ary primitive recursive relation relative to A for some $k \in \mathbb{N}$. Then there exists an $\exists$ -rudimentary formula ϕ relative to A with context $x_1, \dots, x_k, y$ that defines 1_R . Thus $[\phi]_1^y$ defines R . Hence R is $\exists$ -rudimentary relative to A . This completes the proof of (2). $\square$

14. TURING-POST PROGRAMS AND EXISTENTIAL RUDIMENTARY FORMULAS

We fix a subset $A \subseteq \mathbb{N}$.

Definition 14.1. Let S be a state of a Turing-Post machine with oracle for A . Let (l, r) be a pair of numbers such that the following assertions hold.

- (1) The string of symbols in the cells to the left of the head forms the binary expansion of l .
- (2) The string of symbols in the cells to the right of the head, including the cell under the head and with the largest infinite block of trailing zeros removed, forms the reversed binary expansion of r .

Then the number $[l, r]$ is *Wang encoding* of S . We denote it by $W(S)$.

Theorem 14.2. There exists a function Γ that maps Turing-Post programs with oracle for A to $\exists$ -rudimentary formulas relative to A such that the following assertions hold.

- (1) Let $\mathcal{P}$ be a Turing-Post program with oracle for A . Then $\Gamma(\mathcal{P})$ is a formula with context x_t, x_m, y_{pc}, y_m .
- (2) For all $t, i_m, o_{pc}, o_m \in \mathbb{N}$ the sentence

$$\Gamma(\mathcal{P})(t, i_m, o_{pc}, o_m)$$

holds in $\mathfrak{N}$ if and only if the execution of $\mathcal{P}$ on the Turing-Post machine with oracle for A started from the state with Wang encoding i_m does not halt before step t and after this step the state of the machine is encoded by o_m and the program counter stores o_{pc} .

For the proof we need the following result.

Lemma 14.2.1. For each $n \in \mathbb{N}$ let $t(n)$ be a number such that the binary expansion of n starts with

$$\underbrace{1 \dots 1}_t 0$$

Then t is primitive recursive function.

Proof of the lemma. Consider binary relation

$$\text{trailingOnes} = \left\{ (n, l) \in \mathbb{N}^2 \mid 2^{l+1} \mid n \div (2^{l+1} - 1) \right\}$$

Note that $(n, l) \in \text{trailingOnes}$ if and only if the binary expansion of n starts with $\underbrace{1 \dots 1}_l$. Observe that

$$t(n) = \mu_{l \leq n} \left[(n, l) \notin \text{trailingOnes} \right]$$

for every $n \in \mathbb{N}$. This completes the proof. $\square$

Proof of the theorem. We need some helper functions that describe operations of the Turing-Post machine with oracle for A executing $\mathcal{P}$.

We define a function oracle_A by formula

$$\text{oracle}_A(i_{pc}, i_m) = \begin{cases} i_{pc} + 1 & \text{if } t(i_m[1]) = 0 \\ i_{pc} + 1 & \text{if } t(i_m[1]) > 0 \text{ and } \mathbb{1}_A(t(i_m[1]) \div 1) = 0 \\ i_{pc} + 2 & \text{otherwise} \end{cases}$$

for every $i_{pc}, i_m \in \mathbb{N}$. Suppose that i_m is the Wang encoding of a state and i_{pc} is the content of the program counter of the machine. Then $\text{oracleStep}_A(i_{pc}, i_m)$ is the content of the program counter after the execution of the oracle instruction for A .

We define a function $\text{nextProgCounter}_{\mathcal{P}}$ by formula

$$\text{nextProgCounter}_{\mathcal{P}}(i_{pc}, i_m) = \begin{cases} i_{pc} + 1 & \text{if } i_{pc} < \text{lh}(\mathcal{P}) \text{ and } \mathcal{P}[i_{pc}] = \text{PRINT } 0, \text{PRINT } 1 \\ i_{pc} + 1 & \text{if } i_{pc} < \text{lh}(\mathcal{P}) \text{ and } \mathcal{P}[i_{pc}] = \text{MOVE LEFT} \\ i_{pc} + 1 & \text{if } i_{pc} < \text{lh}(\mathcal{P}) \text{ and } \mathcal{P}[i_{pc}] = \text{MOVE RIGHT} \\ i_{pc} + 1 & \text{if } i_{pc} < \text{lh}(\mathcal{P}) \text{ and } \mathcal{P}[i_{pc}] = \text{IS ZERO and } 2 \mid i_m[1] \\ i_{pc} + 2 & \text{if } i_{pc} < \text{lh}(\mathcal{P}) \text{ and } \mathcal{P}[i_{pc}] = \text{IS ZERO and } 2 \nmid i_m[1] \\ n & \text{if } i_{pc} < \text{lh}(\mathcal{P}) \text{ and } \mathcal{P}[i_{pc}] = \text{GO TO } n \text{ for } n \in \mathbb{N} \\ \text{oracle}_A(i_{pc}, i_m) & \text{if } i_{pc} < \text{lh}(\mathcal{P}) \text{ and } \mathcal{P}[i_{pc}] = \text{ORACLE}_A \\ \text{lh}(\mathcal{P}) + 1 & \text{otherwise} \end{cases}$$

for every $i_{pc}, i_m \in \mathbb{N}$. Suppose that i_m is the Wang encoding of a state and i_{pc} is the content of the program counter of the machine. We have two cases.

- If $i_{pc} < \text{lh}(\mathcal{P})$, then $\text{nextProgCounter}_{\mathcal{P}}(i_m, i_{pc})$ returns the content of the program counter after executing $\mathcal{P}[i_{pc}]$ on the machine in the state described by i_m .
- If $\text{lh}(\mathcal{P}) \leq i_{pc}$, then $\text{nextProgCounter}_{\mathcal{P}}(i_m, i_{pc}) = \text{lh}(\mathcal{P}) + 1$.

We define a function $\text{nextState}_{\mathcal{P}}$ by formula

$$\text{nextState}_{\mathcal{P}}(i_{pc}, i_m) = \begin{cases} [i_m[0], i_m[1] \div 1] & \text{if } i_{pc} < \text{lh}(\mathcal{P}) \text{ and } \mathcal{P}[i_{pc}] = \text{PRINT } 0 \text{ and } 2 \nmid i_m[1] \\ [i_m[0], i_m[1] + 1] & \text{if } i_{pc} < \text{lh}(\mathcal{P}) \text{ and } \mathcal{P}[i_{pc}] = \text{PRINT } 1 \text{ and } 2 \mid i_m[1] \\ i_m & \text{otherwise} \end{cases}$$

for every $i_{pc}, i_m \in \mathbb{N}$. Suppose that i_m is the Wang encoding of a state and i_{pc} is the content of the program counter the machine pointing to an instruction in $\mathcal{P}$. Then $\text{nextState}_{\mathcal{P}}(i_m, i_{pc})$ returns the Wang encoding of the machine state after executing $\mathcal{P}[i_{pc}]$ on the machine in the state described by i_m .

We define function $\text{next}_{\mathcal{P}}$ by formula

$$\text{next}_{\mathcal{P}}(j, s) = [\text{nextProgCounter}_{\mathcal{P}}(s[0], s[1]), \text{nextState}_{\mathcal{P}}(s[0], s[1])]$$

for every $j, s \in \mathbb{N}$.

Finally, the function $\text{snapshot}_{\mathcal{P}}$ is obtained by primitive recursion from $\text{next}_{\mathcal{P}}$ and the function given by formula $i_m \mapsto [0, i_m]$ for every $i_m \in \mathbb{N}$. Suppose that i_m is the Wang encoding of a state of the machine and $t \in \mathbb{N}$. Assume that the execution of $\mathcal{P}$ starts in a state encoded by i_m with the program counter pointing to the first instruction of $\mathcal{P}$. If the execution does not halt before step t , then

$$\text{snapshot}_{\mathcal{P}}(t, i_m)[0], \text{snapshot}_{\mathcal{P}}(t, i_m)[1]$$

are, respectively, the content of program counter and the Wang encoding of the state of the machine after t -th step of the execution. If the execution halts before step t , then

$$\text{snapshot}_{\mathcal{P}}(t, i_m)[0] = \text{lh}(\mathcal{P}) + 1$$

These facts follow from descriptions of $\text{nextProgCounter}_{\mathcal{P}}$ and $\text{nextState}_{\mathcal{P}}$.

Now we prove the theorem. Note that oracle_A is primitive recursive relative to A . Hence $\text{nextProgCounter}_{\mathcal{P}}$ is primitive recursive relative to A and $\text{nextState}_{\mathcal{P}}$ is primitive recursive. Thus $\text{snapshot}_{\mathcal{P}}$ is primitive recursive relative to A . Next we define a relation

$$\{(t, i_m, o_{pc}, o_m) \mid \text{snapshot}_{\mathcal{P}}(t, i_m)[0] = o_{pc} \leq \text{lh}(\mathcal{P}) \text{ and } \text{snapshot}_{\mathcal{P}}(t, i_m)[1] = o_m\}$$

This relation is primitive recursive relative to A . By Corollary 13.8 there exists an $\exists$ -rudimentary formula $\Gamma(\mathcal{P})$ relative to A with context x_t, x_m, y_{pc}, y_m that defines the relation above. According to description of $\text{snapshot}_{\mathcal{P}}$ the formula $\Gamma(\mathcal{P})$ satisfies the statement. $\square$

Theorem 14.3. *Each Turing-Post computable function relative to A is $\exists$ -rudimentary relative to A .*

For the proof we need the following results.

Lemma 14.3.1. *For each $k \in \mathbb{N}$ there exists a primitive recursive function that maps $\vec{n} \in \mathbb{N}^k$ into the Wang encoding of the tape of a Turing-Post machine storing $\vec{n}$.*

Proof of the lemma. Left for the reader as an exercise. $\square$

Lemma 14.3.2. *Let U_l be the state of the Turing-Post machine in which the tape stores a single block of l consecutive 1s and the head is pointing to the beginning of the block for some $l \in \mathbb{N}$. Then the function*

$$n \mapsto \begin{cases} l & \text{if } n = W(U_l) \text{ for some } l \in \mathbb{N} \\ 0 & \text{otherwise} \end{cases}$$

is primitive recursive.

Proof of the lemma. We have

$$W(U_l) = [0, 2^{l+1} - 1]$$

for every $l \in \mathbb{N}$. Thus $n = W(U_l)$ for some $l \in \mathbb{N}$ if and only if $l = e_2(n[1] + 1) \div 1$ and $n = [0, 2^{l+1} - 1]$. This implies that the function in the statement is given by

$$n \mapsto \begin{cases} e_2(n[1] + 1) \div 1 & \text{if } n = [0, 2^{e_2(n[1] + 1) \div 1} - 1] \\ 0 & \text{otherwise} \end{cases}$$

and hence is primitive recursive. $\square$

Proof of the theorem. Let f be a k -ary Turing-Post computable function relative to A for some $k \in \mathbb{N}$ and let $\mathcal{P}$ be a Turing-Post program with oracle for A that computes f . For shorthand we denote $\text{lh}(\mathcal{P})$ by $l_{\mathcal{P}}$. By Lemma 14.3.1 and Corollary 13.8 there exists an $\exists$ -rudimentary formula η with context $x_1, \dots, x_k, z$ such that for every $\vec{n} \in \mathbb{N}^k$ the sentence

$$\eta(n_1, \dots, n_k, i_m)$$

holds in $\mathfrak{N}$ if and only if i_m is the Wang encoding of the Turing-Post machine with oracle for A that stores $\vec{n}$. Similarly by Lemma 14.3.2 and Corollary 13.8 there exists $\exists$ -rudimentary formula ζ with context y_m, y such that for every $(n, l) \in \mathbb{N}$ the sentence

$$\zeta(n, l)$$

holds in $\mathfrak{N}$ if and only if $n = U_l$.

Without loss of generality we may assume that variables

$$x_1, \dots, x_k, z, x_t, x_m, y_{pc}, y_m, y$$

are distinct and correctly ordered according to the well-order on $\mathbf{Ar}_A$. Next we consider the formula

$$\phi_f \equiv \exists_{x_m} \exists_{x_t} \exists_{y_m} ([\eta]_{x_m}^z \wedge [\Gamma(\mathcal{P})]_{l_P}^{y_{pc}} \wedge \zeta)$$

Theorem 14.2 implies that ϕ_f is an $\exists$ -rudimentary formula relative to A with context $x_1, \dots, x_k, y$ such that for every $\vec{n} \in \mathbb{N}^k$ and $l \in \mathbb{N}$ the sentence

$$\phi_f(n_1, \dots, n_k, l)$$

holds in $\mathfrak{N}$ if and only if $f(\vec{n}) = l$. This proves that f is $\exists$ -rudimentary relative to A . $\square$

REFERENCES